
Agentic Harness Engineering

由可观测性驱动的 Coding Agent Harness 自动演化

Jiahang Lin^{1*†}, Shichun Liu^{1*†}, Chengjun Pan^{2*†}, Lizhi Lin³, Shihan Dou¹,
Zhiheng Xi¹, Xuanjing Huang¹, Hang Yan³, Zhenhua Han^{3†}, Tao Gui^{1†}, Yu-Gang Jiang¹

¹ 复旦大学 ² 北京大学 ³ 上海奇迹智锋科技有限公司

 [china-qijizhifeng/agentic-harness-engineering](https://github.com/china-qijizhifeng/agentic-harness-engineering)

原文: arXiv:2604.25850 (中文翻译版)

摘要

Harness 如今已成为智能体性能的核心环节，它中介了模型与工具、执行环境之间的交互方式。然而 harness 工程至今仍是一门手工技艺，因为将其自动化面临三重困难：可编辑组件构成的异构动作空间、把可操作信号淹没其中的海量轨迹，以及效果难以归因的编辑。本文提出 **Agentic Harness Engineering (AHE)**，一个通过三根相互匹配的可观测性支柱来应对上述挑战的闭环：❶ 组件可观测性为每一个可编辑的 harness 组件赋予文件级表示，使动作空间显式且可回滚；❷ 经验可观测性把数以百万计的原始轨迹 token 蒸馏为分层、可逐级下钻的证据语料，使演化中的智能体真正消化得了；❸ 决策可观测性让每一次编辑都附带一条自声明的预测，并在下一轮用任务级结果加以验证。三根支柱共同把每次编辑变成一份可证伪的契约，从而让 harness 演化能够自主推进，而不至于退化为试错。实验表明，十轮 AHE 迭代把 Terminal-Bench 2 上的 pass@1 从 69.7% 提升到 77.0%，超过人工设计的 harness Codex (71.9%) 以及自演化基线 ACE 与 Training-Free GRPO。冻结后的 harness 无需重新演化即可迁移：在 SWE-bench-verified 上，它比种子少 12% 的 token 取得了最高的总体成功率；在 Terminal-Bench 2 上，它在三个不同模型家族上带来 +5.1 至 +10.1 pp 的跨家族增益，表明演化出的组件编码的是通用工程经验，而非针对特定基准的调优。消融进一步把增益定位到工具、中间件和长期记忆，而非系统提示词。

* 同等贡献。† 通讯作者: hzhua201@gmail.com, tgui@fudan.edu.cn。‡ 在上海奇迹智锋科技有限公司实习期间完成的工作。

这些结果表明，可观测性驱动的演化是一条切实可行的路径，可以让 coding agent 的 harness 随其基座模型持续共同改进。

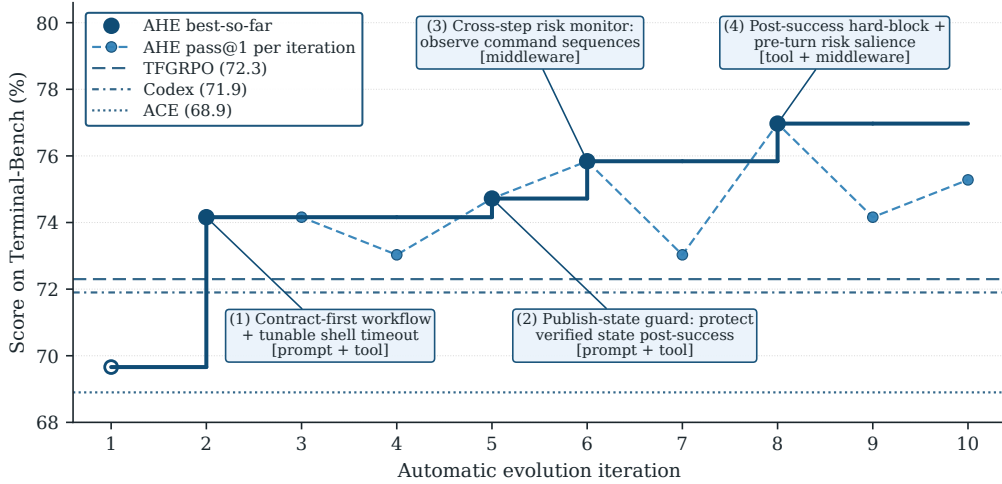


图 1: AHE 把一个仅有 bash 的种子 harness 演化到超越 Terminal-Bench 2 上所有人工设计与自演化基线。三个角色智能体共用同一基座模型，从而把增益隔离到 harness 编辑本身，而非分析器或编辑器的能力差异。

1 引言

coding agent 正被越来越多地部署到长程软件工程任务上，在真实代码仓库的问题 (issue) 解决 [14, 46, 7] 以及多步终端工作流 [21] 上取得了可度量的进展。在实践中，这类进展不仅依赖底层的语言模型，同样依赖围绕其构建的工程组件：塑造工作风格的系统提示词、把文件系统与 shell 暴露出来的工具，以及控制上下文、执行与恢复的中间件。这一组位于模型之外、可编辑的组件被统称为智能体的 *harness* [29, 18, 42, 45, 33, 31]。

即使固定基座模型，harness 设计也会实质性地改变长程编码基准上的任务完成情况 [40, 42]，这使 harness 工程成为改进 coding agent 的一级杠杆。而且，最优的 harness 是模型相关的：为某一基座模型调优的 harness 在另一模型上往往表现不佳，必须随着基座模型的更换而重新适配。在当前实践中，这种适配由人工完成——开发者检查轨迹、识别反复出现的失败模式，并跨提示词、工具、中间件与技能手工编写修改。然而，随着基座模型快速演进 [39, 38, 44, 6, 35, 36]，这一人工闭环难以跟上节奏，在模型能力与释放该能力所需的 harness 之间造成了不断扩大的差距 [33]。

一个直观的方向是用一个演化智能体来自动化这一循环，由它基于经验来优化 harness 组件 [1, 49, 4]。然而，现有方法中鲜有能联合演化全部可编辑组件的 [16]；大多数只聚焦于单一组件，通常是提示词 [32, 50, 20]、技能 [19, 43] 或上下文中的操作手册 (playbook) [49]。端到端地联合演化多个组件面临两个结构性障碍：冗长且无结构的轨迹几乎提供不了可付诸行动的信号；而高度耦合的 harness 框架使得提示词之外的修改极易出错。

这使得智能体驱动的 harness 演化的核心问题仍然悬而未决：演化智能体如何才能联合地且稳定地演化 *coding agent harness* 的全部可编辑组件？

我们的核心洞见是：该问题的瓶颈在于可观测性，而不在于智能体的能力——一旦演化智能体在一个清晰的动作空间之上获得了结构化的上下文，它就能可靠地收敛到更好的 harness 设计 [34, 53]。我们在 **Agentic Harness Engineering (AHE)** (图 2) 中实现了这一点，它是一个由三大可观测性支柱驱动的闭环：❶ 组件可观测性，通过一个解耦的 harness 把七类可编辑组件以文件形式暴露出来，使每一种失败模式都能干净地映射到单一的组件类别；❷ 经验可观测性，通过从数百万原始轨迹 token 中蒸馏出的分层、可逐级下钻的证据语料，使演化者消费的是结构化的根因而非原始日志；以及 ❸ 决策可观测性，通过一份变更清单为每一次修改配上自我声明的预测，随后用下一轮的任务级结果对其加以验证，从而使每一次修改都成为一份可证伪的契约，无效的修改则以文件粒度被回滚。

我们在 Terminal-Bench 2[21] 上对 AHE 进行了实证验证：十轮迭代把 pass@1 从 69.7% 提升到 77.0%，超过了人工设计的 Codex [25] 以及自演化基线 ACE [49] 与 Training-Free GRPO [4]。无需进一步演化，冻结后的 harness 即可迁移到 SWE-bench-verified [14]；在另外三个基座模型家族上，它带来了 +5.1 到 +10.1 个百分点的一致 pass@1 提升，其中距离饱和更远的基座模型收益最大，这表明 AHE 编码了越是未饱和的模型越依赖的协调模式。组件消融进一步定位了这一增益的来源：工具、中间件与长期记忆各自单独都能承载这一提升，而仅有系统提示词时反而出现回归，这说明事实性的 harness 结构能够跨任务、跨模型迁移，而散文层面的策略则不能。

本文做出三点贡献：

- 我们为 coding agent 形式化了智能体驱动的 *harness* 演化问题，并提出 **AHE**：它将跨组件、跨轨迹与跨决策的可观测性确立为设计枢纽，并通过三大可观测性支柱把每一次 harness 修改都变成可证伪的、文件级的契约——一个解耦的组件基底、一条分层的轨迹蒸馏流水线，以及一份其自我声明的预测由下一轮任务差值加以验证的变更清单。
- 我们通过实验表明，AHE 把 Terminal-Bench 2 上的 pass@1 从 69.7% 提升到 77.0%，超越了人工设计的与自动化的基线，并产出了一个能够跨基准、跨基座模型家族迁移的冻结 harness。
- 我们的分析揭示了智能体驱动演化的两个局限：harness 组件之间以非可加的方式相互作用，因此叠加多个有效修改会使总体增益封顶；以及该闭环的自归因对于修复是可靠的，却对回归视而不见，这指明了回归的前瞻预判是未来自演化闭环最清晰的改进方向。

2 相关工作

2.1 面向 coding agent 的 harness 工程与评测

harness 工程指的是设计模型周边系统的实践，包括其工具、接口、记忆、执行约束与反馈闭环，它们共同塑造了智能体在长程任务上能够做什么 [29, 18, 40, 3, 33, 31]。具体而言，harness 中介了模型如何感知环境并在其上行动：它暴露出工具增强推理所依托的动作与观测接口 [3]、用于仓库导航、文件编辑与命令执行的定制智能体—计算机接口 [45]，以及使长程运行保持可复现的沙箱化执行与编排支持 [42]。

为验证这类系统是否真的有帮助，coding agent 的评测沿两条轴线并行走向成熟：任务时程与环境真实性。其覆盖范围从关注污染与时效性控制的短程函数级基准 [52, 12]，经由仓库规模的可执行补丁求解 [14, 46, 7]，一直延伸到考察长程、真实执行的多小时终端驱动工作流 [22, 5, 21]。与之并行的基础设施路线围绕这些基准封装了可执行运行时与验证器 [28, 13, 47]，其对可复现、可追溯、可验证执行的重视，正是 AHE 所依托的观测系统的直接动机。

2.2 LLM 智能体的自动化优化

自动化智能体优化的各种方法，差别在于优化器观测到什么证据、以及它能够编辑什么。一些工作通过按回合 (episodic) 的批评与反思来修订智能体自身的输出 [20, 32, 9]。另一些工作则以提示词与指令为目标 [15]：结构化的操作手册 [49]、语义优势先验 [4]、面向多阶段程序的指令与示例联合优化流水线 [27]，以及由帕累托前沿轨迹驱动的反思式更新 [1]。还有一条独立的路线直接编辑程序结构本身，其形式包括技能库 [41]、通过变异演化并打分的程序与智能体档案库 [24, 11]，以及从 rollout 中搜索或学习得到的图结构工作流 [48, 51]。

AHE 把完整的 harness 作为一个组合整体来调优，而不是只调优单一的可编辑面，因此跨组件的权衡对优化器而言是可读的。它同时把人类先验保持在最低限度，将方法论留给优化器从 rollout 中自行发现，而不是由人手工写定。我们将在第 3 节描述实现这些选择的基底、轨迹分析与迭代过程。

3 方法

AHE 把 harness 优化转化为一个由另一个智能体驱动的闭环：基础模型保持固定，只对显式的 harness 进行编辑。我们的设计原则是，该闭环的每一个阶段都必须是可观测的：AHE 忠实记录每个阶段产出的产物（一次迭代写入的 harness 组件、它生成的 rollout 轨迹、它提交的编辑决策），并将它们表示为结构化、分层的形式，使另一个智能体可以读取并据以行动。

三个可观测性层次实现了这一原则。**组件可观测性** (§3.1) 由一个解耦的、文件级的 harness 基座实现，它把每一种失败模式映射到单一的组件类别。**经验可观测性** (§3.2) 由一个分层的证据语料实现，它从原始 rollout 中蒸馏而来，并建立索引以支持逐级下

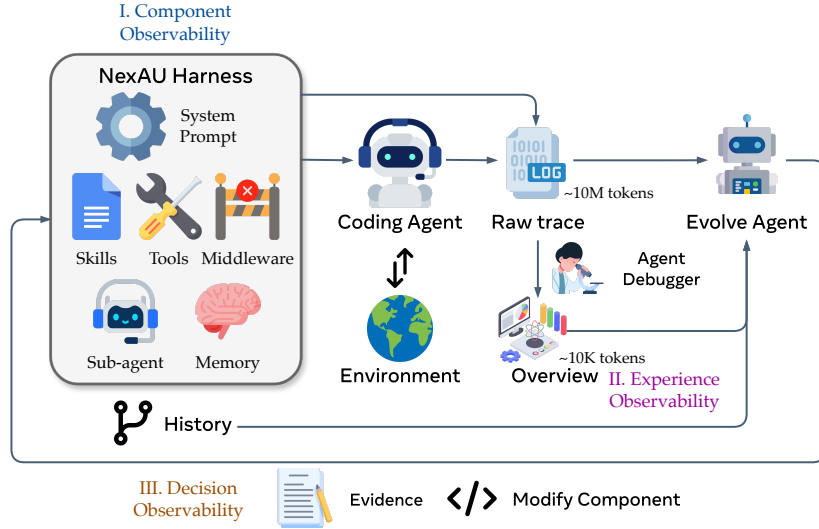


图 2: AHE 流水线把三个可观侧面串联成一个闭环。组件、rollout 经验与编辑决策各自以结构化产物的形式呈现，供另一个智能体读取；而每一次编辑都成为一条可证伪的预测，由下一轮验证。

钻式访问。**决策可观测性** (§3.3) 由一份变更清单实现，它为每一次编辑配上一条自我声明的预测，并由下一轮加以验证。这三个层次共同组成算法 1 所描述的迭代，该迭代一轮接一轮地无人值守运行。

3.1 NexAU: 可编辑、解耦的 harness 基座

我们在 NexAU 框架 [23, 37] 上实例化 harness H 。该框架在单一工作区中，以固定挂载点上的显式文件形式暴露七种正交的组件类型：系统提示词、工具描述、工具实现、中间件、技能、子智能体配置以及长期记忆。这些组件类型之间是松耦合的，因此添加一个中间件不需要编辑系统提示词，添加一个技能也不需要触碰任何工具。

正是这种解耦实现了**组件可观测性**：每一种失败模式都映射到单一的组件类别，从而为 Evolve Agent 提供了干净的动作空间，并把每一次通过率变化定位到一个文件，而不是让它散落在数百行无结构的提示词文字之中。每一次逻辑上的编辑都成为工作区 git 历史上的一次提交，由此免费获得文件级的 diff 与回滚粒度。

我们的种子 harness H_0 刻意保持极简：只有一个 shell 执行工具，没有中间件、没有技能、没有子智能体。若种子已经针对目标基准做过适配，就会污染此后每一次编辑的归因，因为我们将无法分辨增益究竟来自闭环还是来自种子。极简的种子迫使 AHE 添加的每一个组件都必须凭实测的 rollout 来赢得自己的位置。

3.2 Agent Debugger: 分层的轨迹证据

我们使用 harness H 为基准中的每个任务生成 k 条轨迹；这些轨迹中可能包含由 harness 缺陷导致、且可据以采取行动的错误，但它们散落在数百万 token 的原始消息之

中。为了从智能体轨迹中提取洞见并实现**经验可观测性**，我们采用 Agent Debugger [17] 框架：用一个智能体去探索被组织成可导航的、基于文件的环境的轨迹，其中每条轨迹消息各自存放在一个文件里，并通过通用的 shell 与脚本工具访问。同一查询对应的轨迹被放入同一个环境，debugger 被要求分析失败的根因或成功的模式，其结果为每个任务存入一份按任务分析报告。该分析还包含任务的通过/失败状态，用以为 Evolve Agent 提供依据。最后，把每一份报告汇聚成单一文档，形成基准级总览，作为每一次迭代的入口。

除这些报告之外，我们还提供原始轨迹，以备智能体需要核验报告中的论断。轨迹同时以原始形式以及经过轻度处理、去除不必要内容的形式提供。所有这些内容都以文件形式提供，从而支持渐进式披露 [30]，既节省 token，也使智能体能作出更好的决策。

3.3 Evolve Agent：证据驱动、可审计的编辑

Evolve Agent 闭合了 AHE 的闭环。在每一轮中，它读取 Agent Debugger 产出的分层证据语料，决定要添加、修改或移除哪些 harness 组件，把这些编辑应用到工作区，并记录每一次编辑背后的推理。有两条约束支配这些编辑，二者共同实现了**决策可观测性**：每一次编辑都成为一条可证伪的、文件级的论断，被记录在带版本的变更清单中；而下一轮的裁决要么确认它，要么将其回滚。

第一条约束是可控性：Evolve Agent 只能在 harness 工作区内写入，而 runs 目录、tracer、验证器与 LLM 配置均为只读，种子系统提示词（附录 B.1）被标记为不可删除。这些限制阻断了一个不受约束的自我修改者会采取的捷径，例如关闭验证器、替换模型或提高推理预算，从而使记录到的每一分增益都可归因于 harness 编辑。

第二条约束是，每一次变更都必须是证据驱动的，并附带一条被记录下来的预测。每一次编辑都附上一条变更清单条目，其中指明失败证据、推断出的根因、有针对性的修复，以及预测的影响——既包括预期会被修复的任务，也包括存在回归风险的任务；这份变更清单就是闭环的证据账本（见附录 B.2）。在下一轮中，闭环把预测修复集合与预测回归集合同观测到的任务级变化取交集，从而给出针对每一次编辑的裁决。由此，每一次编辑都可被下一次评测证伪，这就用轮次之间可度量的契约取代了以理由为驱动的自我辩护。

算法 1 把三个基座组合进同一次迭代：rollout、清洗、对上一轮变更清单归因并回滚被否决的编辑、蒸馏、编辑、提交。我们对每个任务运行 $k \geq 2$ 次 rollout，使每个任务携带一个通过率信号，这既稳定了 pass@1，也让部分通过的任务成为对比诊断的锚点。归因在蒸馏之前运行，因此它的裁决会落入证据语料之中，并把上一轮变更清单的每一条目约束为一份契约，而非一段说辞。一个一次性的 Explore Agent（附录 B.3）与第 1 轮迭代并行运行，依据 NexAU 源码与公开的 coding agent 参考资料播种少量可复用的技能。这些技能不享有任何特殊保护：从第 2 轮迭代起，Evolve Agent 可以依据实测的 rollout 保留、改进或移除它们。

算法 1 AHE 外层循环。

输入： 种子 harness H_0 、基础模型 M 、基准 D 、每个任务的 rollout 次数 k 、最大迭代次数 N

```
1:  $H_{\text{best}} \leftarrow H_0$ 
2: for  $t = 1$  to  $N$  do
3:    $T_t \leftarrow \text{ROLLOUT}(M, H_{t-1}, D, k)$  ▷ 阶段 1: 每个任务  $k$  次 rollout
4:    $\tilde{T}_t \leftarrow \text{CLEAN}(T_t)$  ▷ 阶段 2: 将轨迹归一化为规范形式
5:   if  $t \geq 2$  then ▷ 阶段 3: 对上一轮变更清单归因, 随后回滚
6:      $V_t \leftarrow \text{ATTRIBUTE}(C_{t-1}, T_{t-1}, T_t)$ 
7:      $H_{t-1} \leftarrow \text{ROLLBACK}(H_{t-1}, V_t)$ 
8:   else
9:      $V_t \leftarrow \emptyset$ 
10:  end if
11:   $R_t \leftarrow \text{AGENTDEBUGGER}(\tilde{T}_t)$  ▷ 阶段 4: 分层蒸馏
12:   $(H_t, C_t) \leftarrow \text{EVOLVE}(H_{t-1}, R_t, V_t)$  ▷ 阶段 5: 工作区编辑 + 新的变更清单
13:   $\text{COMMIT}(H_t, C_t, t)$  ▷ 阶段 6: 在 git 中为该次迭代打标签
14:  if  $\text{PASS@1}(T_t) > \text{PASS@1}(H_{\text{best}})$  then  $H_{\text{best}} \leftarrow H_t$ 
15:  end if
16: end for
17: 返回  $H_{\text{best}}$ 
```

4 实验

我们围绕三个问题组织实证研究：AHE 在既有 harness 设计方法的版图中处于什么位置；它产出的成果能否迁移到其优化目标之外；以及闭环内部究竟是什么在驱动收益。

研究问题

1. RQ1 (§4.2): 为什么要做 agentic harness engineering, 而不是采用人工设计的 harness 或其他自动化方法?
2. RQ2 (§4.3): agentic harness engineering 是否会过拟合到它的优化目标上?
3. RQ3 (§4.4): AHE 内部是什么在驱动收益, 闭环的自归因又有多可靠?

4.1 实验设置

评测。 我们在 Terminal-Bench 2 [21] 的全部 89 个任务上驱动演化, 其难度划分为 4 个 easy、55 个 medium 和 30 个 hard, 并将单任务超时延长至 1 小时。对于跨基准的迁移, 我们在 SWE-bench-verified [14] 上评测 AHE harness, 该基准包含跨七个代码仓库的 500 个任务。我们为每个配置报告两项指标: pass@1 , 即每个任务 k 次 rollout 上二值成功率的均值; 以及 $\text{tokens}/\text{trial}$, 即所有 LLM 调用的提示词 token 与补全 token

之和在每次试验上的均值，以千为单位。因基础设施中止或超时的试验在 pass@1 下计为失败（与官方 terminal-bench 排行榜的口径一致），并从 token 均值中剔除，以免得到被截断的数值。运行时基础设施（框架、调度器、沙箱、追踪器与并发设置）详见附录 A。

模型。 在演化闭环与 §4.2 的主实验中，三个角色智能体 (Code Agent、Agent Debugger 与 Evolve Agent) 共享同一个基座模型：GPT-5.4 [26]，推理档位为 high。对于跨模型迁移 (§4.3)，我们在五个替代基座上重新评测 Code Agent：medium 与 xhigh 推理档位的 GPT-5.4 [26]、qwen-3.6-plus [38, 44]、gemini-3.1-flash-lite-preview [8] 以及 deepseek-v4-flash [6]。

4.2 RQ1: 主要结果

我们在 Terminal-Bench 2 上从仅含 bash 的 NexAU₀ 种子 (§3.1) 出发，运行了一次共十轮迭代的 AHE 演化，历时约 32 小时；其中最优配置被报告为 **AHE**。两个自演化基线 ACE [49] 与 Training-Free GRPO (TF-GRPO) [4] 也从同一个 NexAU₀ 种子出发。

AHE 同时超越人工设计的与自演化的基线。 AHE 在我们的对照面板上超越了每一个基线：三个人工设计的 harness——OpenCode [2]、Terminus-2 [10] 与 Codex [25]，以及两个自演化基线。图 1 显示收益是跨迭代累积的，持续演化把 pass@1 进一步推高到 NexAU₀ 种子之上。

按难度看，唯一的例外是 Hard 档，AHE

在该档略微落后于 Codex。我们把这一差距归因于 AHE 各组件在长程任务上的相互干扰，而非缺失某项能力：只把 AHE 的长期记忆单独换入 NexAU₀ 种子、不引入其他 AHE 组件，就已经在 Hard 上超过了 Codex (§4.4.1)。

仅提示词的自演化错过了承载 AHE 收益的那些组件。 与 ACE 和 TF-GRPO 的差距源于层次上的错配。ACE 蒸馏出智能体在上下文中阅读的自然语言操作手册，而 TF-GRPO 是 GRPO 的一种轨迹反馈变体，用于强化成功的工具调用序列；两种方法都没有把外围脚手架开放给编辑。AHE 则联合演化系统提示词、工具、中间件与长期记忆，而 §4.4.1 表明收益集中在后三个组件上，恰恰是 ACE 与 TF-GRPO 未曾触及的层次。

表 1: Terminal-Bench 2 全部 89 个任务上按官方难度划分的 pass@1。NexAU₀ 是共享的种子；ACE、Training-Free GRPO 与 **AHE** 是叠加在其之上的三种自演化闭环。粗体标出每列最优；并列时全部加粗。

方法	全部 89	简单 4	中等 55	困难 30
人工设计的 harness				
OpenCode	47.2%	75.0%	52.7%	33.3%
Terminus-2	62.9%	75.0%	74.5%	40.0%
Codex	71.9%	75.0%	80.0%	56.7%
从 NexAU₀ 自演化				
NexAU ₀	69.7%	87.5%	78.2%	51.7%
ACE	68.9%	91.7%	78.2%	48.9%
TF-GRPO	72.3%	100.0%	79.4%	55.6%
AHE	77.0%	100.0%	88.2%	53.3%

表 2: SWE-bench-verified 上的跨基准迁移。ACE、Training-Free GRPO (TF-GRPO) 与 **AHE** 共享 **NexAU₀** 种子，仅在各自的自演化闭环上不同；四列全部运行在 GPT-5.4 上。AHE 与两个自演化基线都是在 Terminal-Bench 2 上演化得到的，评测时不做域内再演化。每列粗体标出最优；并列时全部加粗。

仓库	<i>N</i>	成功率 ↑				Tokens <i>k</i> ↓			
		ACE	TF-GRPO	NexAU ₀	AHE	ACE	TF-GRPO	NexAU ₀	AHE
全部	500	74.6%	74.2%	75.2%	75.6%	679	582	526	461
django	231	79.2%	78.8%	79.2%	81.0%	707	583	527	484
sympy	75	69.3%	68.0%	70.7%	70.7%	602	572	494	479
sphinx-doc	44	61.4%	65.9%	68.2%	70.5%	990	848	731	656
matplotlib	34	70.6%	70.6%	73.5%	73.5%	622	530	486	391
scikit-learn	32	93.8%	93.8%	93.8%	87.5%	451	378	307	257
pydata	22	77.3%	77.3%	77.3%	72.7%	563	516	386	338
astropy	22	59.1%	59.1%	54.5%	50.0%	546	470	667	277

4.3 RQ2: 向未见任务与基座模型的迁移

AHE 的 harness 是在 Terminal-Bench 2 上以 GPT-5.4 high 演化得到的。我们把演化后的 harness 原封不动、不再继续演化地迁移到两类目标外设置中，以检验它编码的是通用的 coding agent 经验，还是对该目标的过拟合：一个不同的任务面（SWE-bench-verified）以及三个替代基座模型。

跨基准迁移。 我们在完全相同的基础设施下，于 SWE-bench-verified 上把 AHE harness 与种子及两个自演化基线（NexAU₀、ACE、TF-GRPO）进行对比评测（表 2）。

ACE 与 TF-GRPO 在总体成功率上都跌到 NexAU₀ 种子之下，同时比种子多消耗 11% 到 29% 的 token：ACE 注入的操作手册与 TF-GRPO 强化的轨迹分布，都是在 Terminal-Bench 2 的轨迹上蒸馏出来的，并且在每一次模型调用时都随提示词一同带入，因此在不同的任务面上，这些文本只增加了成本，却没有重塑底层策略。

AHE 则取得了最高的总体成绩，相对种子的增益集中在 django 与 sphinx-doc 这两个规模最大、token 开销最高的仓库上——它们的多步「编辑—验证」循环，正好与 AHE 的工具、中间件和长期记忆在 Terminal-Bench 2 上所压缩的结构相吻合。轻微的回归只出现在三个最小的仓库上，这与小仓库上 pass@1 的方差超过其单仓库增益的情形一致。AHE 还把总体 token 消耗相对 ACE 削减了 32%、相对 TF-GRPO 削减了 21%、相对种子削减了 12%：把行为编码进工具、中间件与记忆之中，避免了仅提示词的基线所付出的逐次调用重新推导成本。

跨模型迁移。 我们在 §4.1 列出的五个替代基座上，对 NexAU₀ 种子与 AHE 分别重新评测。图 3 报告了从 +2.3 到 +10.1 个百分点的五个正向 pass@1 增益。

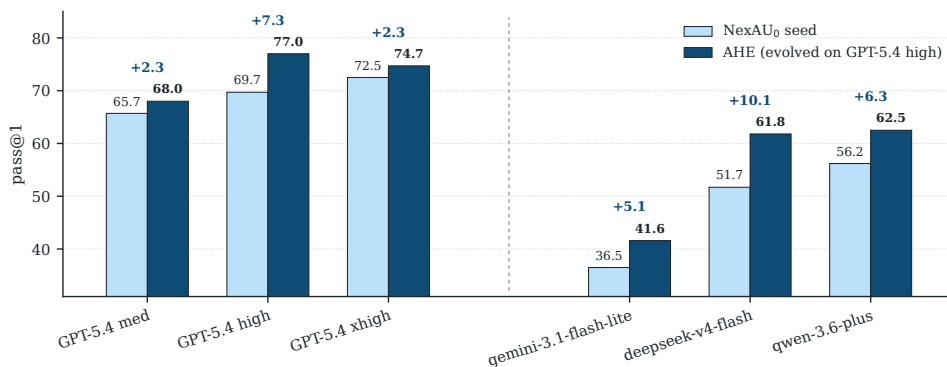


图 3: Terminal-Bench 2 上 89 个任务的跨模型迁移。在 GPT-5.4 high 上演化得到的 AHE 工作区在每个底座模型上重新评测, 不做进一步演化, 并与同一底座上的 NexAU₀ 种子配对比较。所有底座相对种子都有提升, 其中跨家族底座的提升大于同家族底座。

跨模型家族的增益明显大于同家族内部的增益: deepseek-v4-flash 从 51.7% 提升到 61.8%, 增幅 +10.1 个百分点; qwen-3.6-plus 从 56.2% 提升到 62.5%, 增幅 +6.3 个百分点; gemini-3.1-flash-lite-preview 从 36.5% 提升到 41.6%, 增幅 +5.1 个百分点; 三者都高于 GPT-5.4 medium 与 xhigh 上的 +2.3 个百分点。我们对此的解读是: 离饱和更远的基座更依赖 AHE 已固化在工具、中间件与长期记忆中的协同模式, 而更强的基座能以很低的边际成本从自身提示词中重新推导出同样的协同。

在同一家族内部, 其表现是非单调的: medium 上 +2.3 个百分点, high 上 (来自 §4.2) +7.3 个百分点, xhigh 上 +2.3 个百分点。AHE 的步数预算与单任务超时是在演化过程中针对 GPT-5.4 high 拟合的; medium 每一步的时间余量更充裕, 但在原始能力上损失了一个推理档位, 而 xhigh 则使更多试验超出单任务超时, 按我们的 pass@1 约定 (§4.1) 这些都计为失败。两个方向都会削减增益。

最重要的发现是五个增益全部为正: AHE 工作区并不专属于某一家供应商的惯用范式或某一种推理深度。增益的幅度跟随的是演化的工作点, 而非基座的原始能力, 因此我们把这种超时—预算耦合视为一种泛化风险, 并在 [局限性](#) 一节中加以讨论。

4.4 RQ3: 分析

我们沿着 §3 所强调的两个架构选择来分析这一闭环: 分解式组件 (§4.4.1) 与自我声明的归因 (§4.4.2)。

4.4.1 RQ3a: 价值累积在哪些组件上

表 3 在组件层面分解了 AHE 的收益: 每次只隔离出四个被演化层之一——长期记忆、工具、中间件或系统提示词, 同时把其余三层保持在种子的默认状态。四个单组件变体中有三个优于种子; 只有替换系统提示词这一项出现了回归。

每个组件各自负责一类不同的失败面。 记忆增加了 12 条边界情形的经验教训 (性能余量、超限排队时的取消、evaluator 风格的收尾、源码打包布局); 在 Hard 档上这些

表 3: Terminal-Bench 2 上的组件级消融。每一行 “+ X only” 把单个 AHE 组件换入 NexAU₀ 种子；每列最优值以粗体标出。多数单组件替换已经超过种子，且各单组件增益之和超过了完整 AHE 的总增益，说明各组件之间是非可加地相互作用，而非干净地叠加。

变体	全部 89 个任务	简单 4 个任务	中等 55 个任务	困难 30 个任务
NexAU ₀	69.7%	87.5%	78.2%	51.7%
+ 仅长期记忆	75.3%	50.0%	83.6%	63.3%
+ 仅工具	73.0%	75.0%	87.3%	46.7%
+ 仅中间件	71.9%	100.0%	81.8%	50.0%
+ 仅 system_prompt	67.4%	75.0%	78.2%	46.7%
AHE 完整	77.0%	100.0%	88.2%	53.3%

教训使其超过完整的 AHE，而在 Easy 档上它们退化为多余的重叠验证。工具演化成了一个 1364 行的 shell，可从每条命令附近的文件中自动浮现契约提示；在 Medium 档上它与完整 AHE 的差距在 0.9 个百分点以内，而在 Hard 档上其内置的发布守卫会过早地关闭循环。中间件加入了一个 finish 钩子，强制执行一次与 evaluator 同构的收尾检查；在 Easy 档上它通过了每一个任务，而在 Hard 档上它抬高了轮次数量。系统提示词编码了 79 行通用纪律，其可执行性依赖于其余三个组件；单独插入时其总体成绩为 -2.3 个百分点。

各组件之间以非可加的方式相互作用，从而给总体收益设了上限。 三个为正的单组件增益加总为 +11.1 个百分点，而完整 AHE 只有 +7.3 个百分点，并且在 Hard 档上仅含记忆的变体超过了完整 AHE：记忆、中间件与系统提示词都在推动同一种收尾式的验证，因此把它们叠加起来，会在长程任务的预算内把轮次花在冗余的重复检查上。由于 Evolve Agent 优化的是一个由 55 个 Medium 任务主导的总体指标，它收敛到了一个偏向 Medium 的权衡，从而让出了一部分 Hard 档上的记忆效应；我们把考虑组件交互的演化留作未来工作。

4.4.2 RQ3b: 闭环的自归因在多大程度上贴合现实

在每一轮演化中，我们的演化模型都会产出一份变更清单，指明它预期在下一轮中修复哪些 Terminal-Bench 2 任务，以及它标记出哪些任务有回归风险。我们把第 $N-1$ 轮的预测与第 N 轮的真实结果进行比较，在 89 个任务上分别针对修复与回归计算标准的精确率与召回率。

证据驱动的目标定位。 图 4 的修复面板表明，演化模型的目标定位是证据驱动的，而非凭空猜测。跨迭代的修复精确率为 **33.7%**、修复召回率为 **51.4%**，大约是随机预测基线 6.5% 与 10.6% 的 5 倍，因此每一次 harness 编辑都落在一个真实且被智能体预期到的目标上，而不是落在面板中任意的一个子集上。

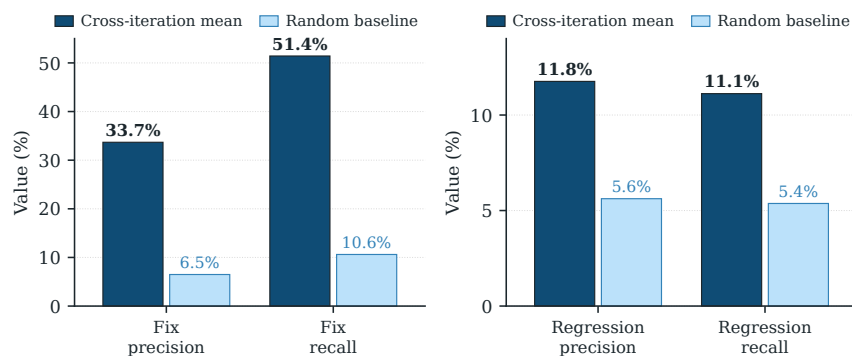


图 4: 在 Terminal-Bench 2 上运行的 GPT-5.4 AHE 闭环的 9 个评测轮次中, 演化模型自我预测的跨迭代平均精确率与召回率, 并与随机预测基线对照。左: 修复预测。右: 回归预测。

回归盲区。 回归面板讲述的却是相反的故事: 跨迭代的回归精确率为 11.8%、回归召回率为 11.1%, 仅约为其随机基线 5.6% 与 5.4% 的 2 倍, 因此大多数即将发生的回归都未被预见。智能体能够论证某次编辑为何应当有帮助, 却无法可靠地指出同一次编辑即将破坏哪些任务, 而这正是 §4.2 中演化曲线出现非单调台阶的原因。弥合这一差距是未来自演化闭环最明确的方向。附录 D 给出了逐轮的细分结果。

5 结论

我们提出了 **Agentic Harness Engineering (AHE)**, 这是一个由可观测性驱动的闭环: 在基座模型保持固定的前提下, 把 coding agent 的 harness 变成一个可学习的适配面。AHE 将各组件以文件形式暴露出来, 把 rollout 蒸馏为分层的证据语料, 并把每一次编辑都绑定到一个可证伪的下一轮预测上; 十轮迭代把 Terminal-Bench 2 上的 pass@1 从 69.7% 提升到 77.0%, 且冻结后的 harness 可迁移到 SWE-bench-verified 以及另外三个模型家族。我们认为, harness 层面的演化是与模型侧训练互补的一条轴线: 它是一个外部化、可审计的适配面, coding agent 的经验可以在其上不断累积。

局限性

本工作研究的是一个前景可观但方差很大的设定, 因此我们结论的适用范围应当据此加以理解。

基准范围。 我们的评测在 Terminal-Bench 2 上驱动演化, 并在 SWE-bench-verified 上考察迁移能力。尽管冻结后的 harness 能够迁移到第二个任务面以及另外三个基座模型家族, 但更广泛的编程语言、仓库规模的部署, 以及人在回路的工作流, 仍未得到检验。

演化工作点。 AHE 的步数预算与单任务超时是在演化过程中针对 GPT-5.4 high 拟合的, 因此跨模型迁移的数字把 harness 的可移植性与工作点耦合混在了一起——在同

一家族内部，增益在不同推理档位之间是非单调的 (§4.3)。要厘清这两个因素，需要在多个工作点下重新运行整个闭环。

自修改治理。 AHE 把编辑限定在一个工作区内，把每一处改动都记录到带版本的变更清单中进行归因，并以文件粒度回滚无效的编辑，但它并未提供一套完整的护栏体系。长程的 harness 清理与更强的滥用防范仍不完善，因此 AHE 应被视为一个受控的研究原型，而非一个完全成熟的自主自我改进系统。

参考文献

- [1] Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alex Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. Gega: Reflective prompt evolution can outperform reinforcement learning. In *The Fourteenth International Conference on Learning Representations*, October 2025. URL <https://openreview.net/forum?id=RQm2KQTM5r>.
- [2] Anomaly. Opencode: The open source coding agent., 2025. URL <https://github.com/anomalyco/opencode>.
- [3] Anthropic. Claude-code, 2025. URL <https://github.com/anthropics/claude-code>.
- [4] Yuzheng Cai, Siqi Cai, Yuchen Shi, Zihan Xu, Lichao Chen, Yulei Qin, Xiaoyu Tan, Gang Li, Zongyi Li, Haojia Lin, Yong Mao, Ke Li, and Xing Sun. Training-free group relative policy optimization, October 2025. URL <http://arxiv.org/abs/2510.08191>.
- [5] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Aleksander Madry, and Lilian Weng. Mle-bench: Evaluating machine learning agents on machine learning engineering. In *The Thirteenth International Conference on Learning Representations*, October 2024. URL <https://openreview.net/forum?id=6s5uXNWGIh>.
- [6] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, April 2026. URL https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf.
- [7] Xiang Deng, Jeff Da, Edwin Pan, Yannis Y. He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa R. Kundurthy, Sean M. Hendryx, Zifan Wang, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. Swe-bench pro: Can

- ai agents solve long-horizon software engineering tasks? October 2025. URL <https://openreview.net/forum?id=9R2iUHhVfr>.
- [8] Google. Gemini-3-1-flash-lite-model-card, March 2026. URL <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-1-Flash-Lite-Model-Card.pdf>.
- [9] Honglin Guo, Kai Lv, Qipeng Guo, Tianyi Liang, Zhiheng Xi, Demin Song, Qiuyinzhe Zhang, Yu Sun, Kai Chen, Xipeng Qiu, and Tao Gui. Critiq: Mining data quality criteria from human preferences. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16240–16261, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.792. URL <https://aclanthology.org/2025.acl-long.792/>.
- [10] Harbor. Terminus-2, 2026. URL <https://www.harborframework.com/docs/agents/terminus-2>.
- [11] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations*, October 2024. URL <https://openreview.net/forum?id=t9U3LW7JVX>.
- [12] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations*, October 2024. URL <https://openreview.net/forum?id=chfJJYC3iL>.
- [13] Naman Jain, Jaskirat Singh, Manish Shetty, Tianjun Zhang, Liang Zheng, Koushik Sen, and Ion Stoica. R2e-gym: Procedural environment generation and hybrid verifiers for scaling open-weights swe agents. In *Second Conference on Language Modeling*, August 2025. URL <https://openreview.net/forum?id=7evvwdo3z#discussion>.
- [14] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, October 2023. URL <https://openreview.net/forum?id=VTF8yNQM66>.
- [15] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling

- declarative language model calls into self-improving pipelines, October 2023. URL <http://arxiv.org/abs/2310.03714>.
- [16] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, March 2026. URL <http://arxiv.org/abs/2603.28052>.
 - [17] Lizhi Lin. Agent debugger: Understanding agent trajectory with agentic workflows - dawning road, February 2026. URL <https://dawning-road.github.io/blog/agent-debugger>.
 - [18] Ryan Lopopolo. Harness engineering: Leveraging codex in an agent-first world, February 2026. URL <https://openai.com/zh-Hans-CN/index/harness-engineering/>.
 - [19] Ziyu Ma, Shidong Yang, Yuxiang Ji, Xucong Wang, Yong Wang, Yiming Hu, Tongwen Huang, and Xiangxiang Chu. Skillclaw: Let skills evolve collectively with agentic evolver, April 2026. URL <http://arxiv.org/abs/2604.08377>.
 - [20] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Thirty-Seventh Conference on Neural Information Processing Systems*, November 2023. URL <https://openreview.net/forum?id=S37h0erQLB>.
 - [21] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heine-man, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking

- agents on hard, realistic tasks in command line interfaces, January 2026. URL <http://arxiv.org/abs/2601.11868>.
- [22] Samuel Miserendino, Michele Wang, Tejal Patwardhan, and Johannes Heidecke. Swe-lancer: Can frontier llms earn \$1 million from real-world freelance software engineering? In *Forty-Second International Conference on Machine Learning*, June 2025. URL <https://openreview.net/forum?id=xZXhFg43EI>.
- [23] Nex-AGI. Nexau (au for agent universe), a general-purpose agent framework for building intelligent agents with tool capabilities., 2025. URL <https://github.com/nex-agi/NexAU>.
- [24] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery, June 2025. URL <http://arxiv.org/abs/2506.13131>.
- [25] OpenAI. Codex cli, 2025. URL <https://developers.openai.com/codex/cli>.
- [26] OpenAI. Introducing gpt-5.4, March 2026. URL <https://openai.com/index/introducing-gpt-5-4/>.
- [27] Krista Opsahl-Ong, Michael J Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9340–9366, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.525. URL <https://aclanthology.org/2024.emnlp-main.525/>.
- [28] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym. In *Forty-Second International Conference on Machine Learning*, June 2025. URL <https://openreview.net/forum?id=Cq1BNvHx74>.
- [29] Prithvi Rajasekaran. Harness design for long-running application development, March 2026. URL <https://www.anthropic.com/engineering/harness-design-long-running-apps>.
- [30] Prithvi Rajasekaran, Ethan Dixon, Carly Ryan, Jeremy Hadfield, Rafi Ayub, Hannah Moran, Cal Rueb, Connor Jennings, Molly Vorwerck, Stuart Ritchie, and Maggie Vo. Effective context engineering for ai

- agents, September 2025. URL <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>.
- [31] Nous Research. Hermes agent — the agent that grows with you, 2026. URL <https://hermes-agent.nousresearch.com/>.
- [32] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R. Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Thirty-Seventh Conference on Neural Information Processing Systems*, November 2023. URL <https://openreview.net/forum?id=vAElhFcKW6>.
- [33] Peter Steinberger. Openclaw — personal ai assistant, February 2026. URL <https://openclaw.ai/>.
- [34] Rich Sutton. The bitter lesson, March 2019. URL https://www.cs.utexas.edu/~eunsol/courses/data/bitter_lesson.pdf.
- [35] Kimi Team. Kimi k2.6 tech blog: Advancing open-source coding, April 2026. URL <https://www.kimi.com/blog/kimi-k2-6>.
- [36] Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, S. H. Cai, Yuan Cao, Y. Charles, H. S. Che, Cheng Chen, Guanduo Chen, Huarong Chen, Jia Chen, Jiahao Chen, Jianlong Chen, Jun Chen, Kefan Chen, Liang Chen, Ruijue Chen, Xinhao Chen, Yanru Chen, Yanxu Chen, Yicun Chen, Yimin Chen, Yingjiang Chen, Yuankun Chen, Yujie Chen, Yutian Chen, Zhirong Chen, Ziwei Chen, Dazhi Cheng, Minghan Chu, Jialei Cui, Jiaqi Deng, Muxi Diao, Hao Ding, Mengfan Dong, Mengnan Dong, Yuxin Dong, Yuhao Dong, Angang Du, Chenzhuang Du, Dikang Du, Lingxiao Du, Yulun Du, Yu Fan, Shengjun Fang, Qiulin Feng, Yichen Feng, Garimugai Fu, Kelin Fu, Hongcheng Gao, Tong Gao, Yuyao Ge, Shangyi Geng, Chengyang Gong, Xiaochen Gong, Zhuoma Gongque, Qizheng Gu, Xinran Gu, Yicheng Gu, Longyu Guan, Yuanying Guo, Xiaoru Hao, Weiran He, Wenyang He, Yunjia He, Chao Hong, Hao Hu, Jiayi Hu, Yangyang Hu, Zhenxing Hu, Ke Huang, Ruiyuan Huang, Weixiao Huang, Zhiqi Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yu Jing, Guokun Lai, Aidi Li, C. Li, Cheng Li, Fang Li, Guanghe Li, Guanyu Li, Haitao Li, Haoyang Li, Jia Li, Jingwei Li, Junxiong Li, Lincan Li, Mo Li, Weihong Li, Wentao Li, Xinhang Li, Xinhao Li, Yang Li, Yanhao Li, Yiwei Li, Yuxiao Li, Zhaowei Li, Zheming Li, Weilong Liao, Jiawei Lin, Xiaohan Lin, Zhishan Lin, Zichao Lin, Cheng Liu, Chenyu Liu, Hongzhang Liu, Liang Liu, Shaowei Liu, Shudong Liu, Shuran Liu, Tianwei Liu, Tianyu Liu, Weizhou Liu, Xiangyan Liu, Yangyang Liu, Yanming Liu, Yibo Liu, Yuanxin Liu, Yue Liu, Zhengying Liu, Zhongnuo Liu, Enzhe Lu, Haoyu Lu, Zhiyuan Lu, Junyu Luo, Tongxu Luo, Yashuo Luo, Long Ma, Yingwei Ma, Shaoguang Mao, Yuan Mei, Xin Men, Fanqing Meng, Zhiyong Meng, Yibo Miao, Mingqing Ni, Kun Ouyang, Siyuan Pan, Bo Pang, Yuchao Qian, Ruoyu

Qin, Zeyu Qin, Jiezhong Qiu, Bowen Qu, Zeyu Shang, Youbo Shao, Tianxiao Shen, Zhennan Shen, Juanfeng Shi, Lidong Shi, Shengyuan Shi, Feifan Song, Pengwei Song, Tianhui Song, Xiaoxi Song, Hongjin Su, Jianlin Su, Zhaochen Su, Lin Sui, Jinsong Sun, Junyao Sun, Tongyu Sun, Flood Sung, Yunpeng Tai, Chuning Tang, Heyi Tang, Xiaojuan Tang, Zhengyang Tang, Jiawen Tao, Shiyuan Teng, Chaoran Tian, Pengfei Tian, Ao Wang, Bowen Wang, Chensi Wang, Chuang Wang, Congcong Wang, Dingkun Wang, Dinglu Wang, Dongliang Wang, Feng Wang, Hailong Wang, Haiming Wang, Hengzhi Wang, Huaqing Wang, Hui Wang, Jiahao Wang, Jinhong Wang, Jiuzheng Wang, Kaixin Wang, Linian Wang, Qibin Wang, Shengjie Wang, Shuyi Wang, Si Wang, Wei Wang, Xiaochen Wang, Xinyuan Wang, Yao Wang, Yejie Wang, Yipu Wang, Yiqin Wang, Yucheng Wang, Yuzhi Wang, Zhaoji Wang, Zhaowei Wang, Zhengtao Wang, Zhexu Wang, Zihan Wang, Zizhe Wang, Chu Wei, Ming Wei, Chuan Wen, Zichen Wen, Chengjie Wu, Haoning Wu, Junyan Wu, Rucong Wu, Wenhao Wu, Yuefeng Wu, Yuhao Wu, Yuxin Wu, Zijian Wu, Chenjun Xiao, Jin Xie, Xiaotong Xie, Yuchong Xie, Yifei Xin, Bowei Xing, Boyu Xu, Jianfan Xu, Jing Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinbo Xu, Xinran Xu, Yangchuan Xu, Yichang Xu, Yuemeng Xu, Zelai Xu, Ziyao Xu, Junjie Yan, Yuzi Yan, Guangyao Yang, Hao Yang, Junwei Yang, Kai Yang, Ningyuan Yang, Ruihan Yang, Xiaofei Yang, Xinlong Yang, Ying Yang, Yi Yang, Yi Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Dan Ye, Wenjie Ye, Zhuorui Ye, Bohong Yin, Chengzhen Yu, Longhui Yu, Tao Yu, Tianxiang Yu, Enming Yuan, Mengjie Yuan, Xiaokun Yuan, Yang Yue, Weihao Zeng, Dunyuan Zha, Haobing Zhan, Dehao Zhang, Hao Zhang, Jin Zhang, Puqi Zhang, Qiao Zhang, Rui Zhang, Xiaobin Zhang, Y. Zhang, Yadong Zhang, Yangkun Zhang, Yichi Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yushun Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Chenguang Zhao, Feifan Zhao, Jinxiang Zhao, Shuai Zhao, Xiangyu Zhao, Yikai Zhao, Zijia Zhao, Huabin Zheng, Ruihan Zheng, Shaojie Zheng, Tengyang Zheng, Junfeng Zhong, Longguang Zhong, Weiming Zhong, M. Zhou, Runjie Zhou, Xinyu Zhou, Zaida Zhou, Jinguo Zhu, Liya Zhu, Xinhao Zhu, Yuxuan Zhu, Zhen Zhu, Jingze Zhuang, Weiyu Zhuang, Ying Zou, and Xinxing Zu. Kimi k2.5: Visual agentic intelligence, February 2026. URL <http://arxiv.org/abs/2602.02276>.

- [37] Nex-AGI Team, Yuxuan Cai, Lu Chen, Qiaoling Chen, Yuyang Ding, Liwen Fan, Wenjie Fu, Yufei Gao, Honglin Guo, Pinxue Guo, Zhenhua Han, Zhengfu He, Hanglei Hu, Kai Hu, Shengjia Hua, Tianyu Huai, Baodai Huang, Li Ji, Zhen Jiang, Zhikai Lei, Bufan Li, Jiahang Lin, Lizhi Lin, Jinxiu Liu, Shichun Liu, Ziming Liu, Yuchen Ni, Pengfang Qian, Yujiong Shen, Qingyun Shi, Wentao Shu, Peng Sun, Yiran Suo, Tian Tang, Boyu Tian, Guoteng Wang, Junzhe Wang, Peixin Wang, Zhiheng Xi, Hang Yan, Jie Yang, Zhixiong Yang, Tianchu Yao, Guangze Ye,

- Qianxi Yu, Shuo Zhang, Xinyue Zhang, Yiqi Zhang, Jiarong Zhao, Miao Zheng, Rui Zheng, Enyu Zhou, Jiazheng Zhou, Maosen Zhou, Yuhao Zhou, Tao Gui, Yining Zheng, Xinchu Chen, Jie Zhou, Siyuan Feng, Qin Chen, Liang He, Qi Zhang, Xuanjing Huang, and Xipeng Qiu. Nex-n1: Agentic models trained via a unified ecosystem for large-scale environment construction, December 2025. URL <http://arxiv.org/abs/2512.04987>.
- [38] Qwen Team. Qwen3.6-plus: Towards real world agents, April 2026. URL <https://qwenlm.github.io/blog/qwen3.6/>.
- [39] Xiaomi MiMo Team. Mimo-v2.5-pro, April 2026. URL <https://huggingface.co/XiaomiMiMo/MiMo-V2.5-Pro>.
- [40] Vivek Trivedy. Improving deep agents with harness engineering, February 2026. URL <https://www.langchain.com/blog/improving-deep-agents-with-harness-engineering>.
- [41] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, October 2023. URL <http://arxiv.org/abs/2305.16291>.
- [42] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for ai software developers as generalist agents, April 2025. URL <http://arxiv.org/abs/2407.16741>.
- [43] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning, February 2026. URL <http://arxiv.org/abs/2602.08234>.
- [44] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui,

- Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, May 2025. URL <http://arxiv.org/abs/2505.09388>.
- [45] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-Eighth Annual Conference on Neural Information Processing Systems*, November 2024. URL [https://openreview.net/forum?id=mXpq6ut8J3&referrer=%5Bthe%20profile%20of%20Shunyu%20Yao%5D\(%2Fprofile%3Fid%3D~Shunyu_Yao1\)](https://openreview.net/forum?id=mXpq6ut8J3&referrer=%5Bthe%20profile%20of%20Shunyu%20Yao%5D(%2Fprofile%3Fid%3D~Shunyu_Yao1)).
- [46] John Yang, Carlos E. Jimenez, Alex L. Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R. Narasimhan, Diyi Yang, Sida Wang, and Ofir Press. Swe-bench multimodal: Do ai systems generalize to visual software domains? In *The Thirteenth International Conference on Learning Representations*, October 2024. URL <https://openreview.net/forum?id=riTiq3i21b>.
- [47] Yucheng Zeng, Shupeng Li, Daxiang Dong, Ruijie Xu, Zimo Chen, Liwei Zheng, Yuxuan Li, Zhe Zhou, Haotian Zhao, Lun Tian, Heng Xiao, Tianshu Zhu, Longkun Hao, and Jianmin Wu. Swe-hub: A unified production system for scalable, executable software engineering tasks, February 2026. URL <http://arxiv.org/abs/2603.00575>.
- [48] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xiong-Hui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. Aflow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations*, October 2024. URL <https://openreview.net/forum?id=z5uVAKwmjf>.
- [49] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations*, October 2025. URL <https://openreview.net/forum?id=eC4ygDs02R>.
- [50] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, December 2024. URL <http://arxiv.org/abs/2308.10144>.
- [51] Wangchunshu Zhou, Yixin Ou, Shengwei Ding, Long Li, Jialong Wu, Tiannan Wang, Jiamin Chen, Shuai Wang, Xiaohua Xu, Ningyu Zhang, Huajun Chen, and Yuchen Eleanor Jiang. Symbolic learning enables self-evolving agents, June 2024. URL <http://arxiv.org/abs/2406.18532>.

- [52] Terry Yue Zhuo, Vu Minh Chien, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widayarsi, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen Gong, James Hoang, Armel Randy Zebaze, Xiaoheng Hong, Wen-Ding Li, Jean Kaddour, Ming Xu, Zhihan Zhang, Prateek Yadav, Naman Jain, Alex Gu, Zhoujun Cheng, Jiawei Liu, Qian Liu, Zijian Wang, Binyuan Hui, Niklas Muennighoff, David Lo, Daniel Fried, Xiaoning Du, Harm de Vries, and Leandro Von Werra. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. In *The Thirteenth International Conference on Learning Representations*, October 2024. URL <https://openreview.net/forum?id=YrycTj1lL0>.
- [53] Gregor Zunic. The bitter lesson of agent harnesses, April 2026. URL <https://browser-use.com/posts/bitter-lesson-agent-harnesses>.

A 实验设置：完整细节

本附录在 §4.1 的精简版设置基础上加以扩展，给出形式化的指标定义与运行时基础设施。

种子智能体。 种子配置记为 NexAU_0 ，它是构建在 §3.1 所述 NexAU 框架之上的一个简单 code agent，只向模型暴露 `bash` 工具，没有技能、没有中间件、也没有长期记忆。AHE 外层循环的每一轮迭代都编辑这个工作区，因此所报告的全部增益都是以 NexAU_0 作为共同起点来度量的。

运行时基础设施。 所有运行都使用 §3.1 的 NexAU 框架来实例化 coding agent。Harbor 负责派发任务、隔离每一次 rollout，并验证通过/失败，单任务超时为 3600 秒。每一次 rollout 都在一个全新的 E2B 远程沙箱中运行，因此 shell 的副作用不会在任务之间泄漏。InMemoryTracer 记录轨迹并将其镜像到 Langfuse。Agent Debugger 以并发度 16 执行，单任务超时为 600 秒。

超参数。 表 4 汇总了四个智能体与外层循环的工作点，取自参考运行的配置快照。

Terminal-bench 难度标签。 官方 terminal-bench-2 排行榜⁰ 把这个包含 89 个任务的子集划分为 4 个 easy、55 个 medium 和 30 个 hard 任务。

pass@1。 对于在任务集 D 上、每个任务做 k 次 rollout 的某个配置，令 $r_{i,j} \in \{0, 1\}$ 表示第 j 次 rollout 在任务 i 上的二值奖励。pass@1 分数即为均值

$$\text{pass@1} = \frac{1}{k|D|} \sum_{i=1}^{|D|} \sum_{j=1}^k r_{i,j}. \quad (1)$$

⁰<https://www.tbench.ai/benchmarks/terminal-bench-2>

因基础设施异常（如沙箱崩溃或 API 超时）而终止的试验，不会被剔除，而是计为 $r = 0$ ；这在严格意义上是比丢弃失败试验更苛刻的约定，能让我们的数字与官方 terminal-bench 排行榜保持可比。rollout 次数 k 在不同实验中有所不同；每张表都会明确标注。

Token 成本与 Succ/Mtok。 在 token 成本方面，我们把整个 rollout 中每一次 LLM 调用的 prompt 与 completion 相加计数，并以千为单位报告在完成试验上的均值，记为 Tokens k ；因基础设施中止的试验被排除在外，以避免出现被截断的数字。为了比较那些以精度换成本的配置，我们通过下式把两者结合起来

$$\text{Succ/Mtok} = \frac{\text{pass@1} \times 10^6}{\text{mean tokens per trial}}, \quad (2)$$

即每百万 token 的期望成功次数。正文分别报告 pass@1 与 Tokens k ，以保证每一个维度都清晰可读；表 5 则把它们折算为 SWE-bench-verified 上按代码仓库划分的 Succ/Mtok，由表 2 的 pass@1 与 Tokens k 两列推导得到。

B 提示词与配置

本附录汇总了驱动 AHE 外层循环的各个提示词，以及种子 code agent 的系统提示词。下面五个代码块逐字复现了公开代码仓库 <https://github.com/china-qijizhifeng/agent-harness-engineering> 中相应文件的内容，其版本对应于产出第 4 节实验结果的那次提交。Jinja 风格的 `{{ var }}` 占位符由 harness 在运行时填充。

B.1 Code Agent 种子系统提示词

这是在迭代 1 载入 NexAU₀ 的种子系统提示词。它有意保持极简：一个工具、三条行为规则以及三个运行时注入的变量。迭代 1 之后的每一轮迭代都可以向该文件追加规则，附录 C 中的案例研究追踪了第一次这样的追加。

code_agent_simple/systemprompt.md

```
You solve software tasks in a non-interactive setting. Your only tool is **run_shell_command**: use the
shell to inspect the repo, edit files, run builds/tests, and finish the work. Do not ask the user
questions.
```

- Prefer short replies; use the tool for actions.
- Before commands that delete or overwrite important data, state briefly what they do.
- Long-running processes: use ``is_background: true`` on ``run_shell_command`` (do not use ``&`` in the command string).

```
Date: {{ date }}
```

```
Username: {{ username }}
```

```
Working Dir: {{ working_directory }}
```

B.2 Evolve Agent 提示词

Evolve Agent 的系统提示词编码了第 3 节所述的三条硬性契约：仅限工作区的可控性、证据驱动的变更，以及以变更清单为交付物。它还内嵌了该智能体必须据以推理的目录结构，以及变更清单的 JSON 格式。

```
evolve_agent/evolve_prompt.md

{% set ws = workspace_path if workspace_path is defined else "workspace" %}

You are the NexAU Evolution Engine -- a meta-agent that iterates on a coding agent's harness to maximize **
pass@1** (single-attempt success rate) through evidence-based experimentation. You may modify existing
components or create new ones (tools, middleware, skills, sub-agents, etc.) as needed.

# Core Principles

## 1. Controllability

Only `workspace/` is your playground. Everything else is read-only or off-limits.

- Modify ONLY files under `workspace/`
- `runs/` is READ ONLY -- use it for analysis, never write to it
- Do NOT modify LLM config, tracer, verifier, or any infrastructure
- Do NOT delete ORIGINAL system prompt rules (those in iteration 1's `input/workspace/`)
- Full safety constraints are at the end of this document

## 2. Evidence-Driven

**Every change must be traceable to specific failure evidence.** Do not make changes based on intuition,
speculation, or "best practices" alone.

**Before making any change, you must have:**

1. **Failure evidence** -- which tasks failed, and what specifically went wrong (from analysis reports or
traces)
2. **Root cause** -- why it failed, not just what failed
3. **Targeted fix** -- a change that directly addresses the root cause
4. **Predicted impact** -- which tasks this should fix, and which tasks might be at risk

# Environment

{% if ws != "workspace" %}
> **WORKSPACE PATH**: Your workspace is at `${ws}` instead of `workspace/`. All `workspace/` references
below apply to `${ws}`. Use `${ws}` in file operations, git commands, and the validation command
.
{% endif %}

> **Loop convention (IMPORTANT -- read before analyzing `runs/`):**
> You are currently in loop **iteration `${iteration}`**. Each `runs/iteration_NNN/` folder mixes **two**
generations of work:
> - `input/` holds what **the previous loop (NNN-1)** produced -- this is the workspace that was just
evaluated this loop. The benchmark, analysis, and change_evaluation inside `input/` all describe the **
previous loop's** changes, not yours.
> - `evolve/` holds what **this loop (NNN)** will produce -- your new changes, which the next loop (NNN+1)
will evaluate.
>
> Concretely: when your query says "Iteration `${iteration}` evaluation completed", it means the eval of **
iteration `${iteration} - 1`'s changes** is done (baseline if `${iteration}` = 1). You are now
making changes that will be labeled iteration `${iteration}` and evaluated next loop.

...
```

```

./
|-- {{ ws }}/                                # work_dir = experiment root
|   |-- code_agent.yaml                      # * MODIFY these files
|   |-- systemprompt.md                     # Agent config (tools, middleware, params, sub-agents)
|   |-- LongTermMEMORY.md                   # System prompt (Jinja template)
|   |-- ShortTermMEMORY.md                  # Long-term memory (persistent cross-session knowledge, MODIFIABLE)
|   |-- tool_descriptions/                  # Short-term memory (managed by code agent at runtime, DO NOT MODIFY)
|   |-- tools/                              # Tool YAML definitions
|   |-- middleware/                         # Tool Python implementations
|   |-- skills/                             # Middleware Python implementations
|   |-- sub_agents/                         # Skill packages
|                                           # Sub-agent configs (optional, you may create)
|
|-- runs/                                    # * READ ONLY
|   |-- iteration_NNN/
|       |-- input/                          # Everything this iteration starts with
|           |-- workspace/                  # Workspace being evaluated this loop
|           |-- benchmark/                  # Eval results for the workspace above
|               |-- {timestamp}/
|                   |-- result.json
|                   |-- {task_name}_{id}/
|                       |-- agent/nexau.txt
|                       |-- agent/nexau_in_memory_tracer.cleaned.json
|                       |-- verifier/reward.txt
|               |-- analysis/                # ** Pre-built failure/success analysis (READ THIS FIRST)
|                   |-- overview.md
|                   |-- detail/{task_name}.md
|                   |-- variant_selection.json
|                   |-- change_evaluation.json
|               |-- evolve/                  # YOUR outputs this loop
|                   |-- evolve_summary.md
|                   |-- change_manifest.json
|               |-- variant_N/
|                   |-- workspace/
|                   |-- evolve_trace.json
|
|-- evolution_history.md                     # Cumulative history of all iterations (READ)
-- config_snapshot.yaml                     # Initial config (READ ONLY)
...

# Components

## Available Component Types

| Component | Files | Characteristics | When to use |
|-----|-----|-----|-----|
| **System Prompt** | `workspace/systemprompt.md` | Advisory -- applies to all tasks | Behavioral rules, workflow guidance |
| **Tool Description** | `workspace/tool_descriptions/*.tool.yaml` | Co-located with tool -- model reads when calling | Clarify tool usage, add examples, warn about pitfalls |
| **Tool Implementation** | `workspace/tools/` | Controls tool behavior directly | New capabilities, smarter error handling, output formatting |
| **Middleware** | `workspace/middleware/` + `code_agent.yaml` | Hooks into agent loop pipeline | Intercept/transform at execution level |
| **Skill** | `workspace/skills/` + `code_agent.yaml` | On-demand -- loaded when relevant | Reusable workflow patterns |
| **Sub-Agent** | `workspace/sub_agents/{name}/` + `code_agent.yaml` | Delegated execution -- isolated context | Offload specialized subtask to child agent |
| **Long-Term Memory** | `workspace/LongTermMEMORY.md` | Persistent cross-session knowledge -- MODIFIABLE | Record recurring pitfalls, proven strategies, environment quirks |
| **Short-Term Memory** | `workspace/ShortTermMEMORY.md` | Session-scoped scratch -- DO NOT MODIFY | _(read-only for evolve agent)_ |

```


All component types are equally valid and important. Choose the one that best fits the root cause.

Choosing the Right Component Level

For each failure pattern, consider **all** component types above -- including creating new ones -- before deciding where to fix.

Anti-pattern: If the same failure class persists across 2+ iterations despite fixes at one component level, that level may be the wrong choice. Rollback the ineffective change and re-approach from a different component level.

Registering New Components

Creating a file is NOT enough -- register in `code\_agent.yaml`:

- New tool: create `.tool.yaml` + Python implementation + add entry to `tools:` list
- New middleware: create Python class + add entry to `middlewares:` list with `import:` path and `params:`
- New skill: create `skills/{name}/SKILL.md` folder + add to `skills:` list
- New sub-agent: create `sub\_agents/{name}/agent.yaml` + add to `sub\_agents:` list. Framework **auto-injects** `RecallSubAgent` tool -- do NOT add it manually.

How Code Gets Loaded

The config directory is added to `sys.path` at runtime:

- `binding: tools.file\_tools:read\_file` resolves to `workspace/tools/file\_tools/read\_file.py`
- `import: middleware.long\_tool\_output:LongToolOutputMiddleware` resolves to `workspace/middleware/long\_tool\_output.py`
- `import: middleware.context\_compaction:ContextCompactionMiddleware` resolves to `workspace/middleware/context\_compaction/\_\_init\_\_.py`

LLM Environment Variables

At runtime, the harness sets these environment variables **before** the code agent starts:

Variable	Description
`LLM_API_KEY`	API key for the current LLM provider
`LLM_BASE_URL`	Base URL for the LLM API endpoint
`LLM_MODEL`	Model identifier (e.g. `gpt-5.4`)

All components -- code agent, sub-agents, and middleware -- use these same env vars:

- In agent YAML files: `\${env.LLM\_API\_KEY}`, `\${env.LLM\_BASE\_URL}`, `\${env.LLM\_MODEL}`
- In middleware Python code: `os.environ["LLM\_API\_KEY"]`, etc.

Do NOT hardcode API keys. Always reference environment variables.

Middleware can call LLM

Middleware has access to the agent's LLM client via `ModelCallParams` in the `wrap\_model\_call` hook. Use `LLMCaller` to make side-calls (e.g. summarize context, classify errors, generate dynamic guidance). See the evolution guide skill for full API reference and examples.

Sub-Agents use the same LLM

Sub-agent YAML configs should use `\${env.LLM\_MODEL}` / `\${env.LLM\_BASE\_URL}` / `\${env.LLM\_API\_KEY}` in their `llm\_config`. This automatically gives them the same LLM provider as the parent agent.

For detailed schemas, creation guides, and code examples, read `evolve\_agent/skills/nexau-evolution-guide/SKILL.md`.

Multi-Variant Results (when present)

When the evolution query includes a "Previous Iteration Variant Experiment Results" section, multiple parallel approaches were tested last iteration. Use this signal:

- ****Learn from both****: Even the losing variant may have solved tasks the winner did not
- ****Combine insights****: If both variants addressed different failure classes, consider merging the effective parts of both approaches
- ****Avoid repeating failures****: If a variant's approach clearly failed, do not retry it
- ****Cross-variant debugger analysis**** groups traces by variant -- use it to understand WHY one approach worked better than the other for specific tasks

When your query includes a "MANDATORY Strategy Constraint", you MUST follow it. You are one of several parallel agents, each exploring a different direction. Violating the constraint wastes the exploration budget.

Analysis Approach

> ****[!] MANDATORY: Read `analysis/` first.**** The analysis reports are pre-built summaries of all task failures with root causes already identified. They save you significant time -- do NOT skip them to read raw traces directly.

1. Read `evolution\_history.md` -- understand what's been tried, what worked, what failed
 2. ****Read `runs/iteration\_NNN/input/analysis/overview.md` FIRST**** -- this is your primary information source
 3. ****Read `runs/iteration\_NNN/input/analysis/detail/{task\_name}.md`**** for tasks needing deeper investigation
 4. Only fall back to reading raw `nexau\_in\_memory\_tracer.cleaned.json` when analysis is missing or insufficient -- this should be rare
 5. ****After creating or modifying middleware****, read at least one `agent/nexau.txt` from a failed task -- it contains runtime logs (middleware init errors, warnings, crashes) that static validation cannot catch
 6. Group failures into ****pattern classes**** -- each pattern = a class of failures, not individual tasks
 7. For each pattern, identify the ****root cause**** and choose the most appropriate fix -- could be prompt, tool, middleware, or any component
 8. ****Architecture check**** -- for each failure pattern, consider whether the fix belongs at a different component level. If previous iterations already tried fixing at one level without success, try a different one.
 9. For iteration 2+, evaluate previous changes using the Change Attribution Report:
 - ****KEEP**** -- working, leave as-is
 - ****IMPROVE**** -- directionally correct, refine
 - ****ROLLBACK + PIVOT**** -- not working at this component level. Rollback the change, then re-approach the same failure pattern from a ****different component level****
- **The sole optimization target is pass@1**** -- the probability that a single attempt succeeds. Every change you make should raise pass@1. Timed-out tasks count as failures -- analyze why the agent ran out of time. Only pure infrastructure exceptions (sandbox crash, etc.) can be ignored.

When the experiment runs k>1 rollouts (indicated in the query), use the extra signal to diagnose:

- ****Partial-pass tasks**** (some rollouts pass, some fail) are the most valuable. Compare the passing and failing rollouts of the ***same task***, find the divergence point, and make the successful strategy the ***reliable default***.
- ****pass@k**** gauges capability ceiling but is NOT the target. Your goal is to turn pass@k successes into pass@1 successes by making the winning strategy consistent.

****For iteration 2+**** Compare task results across iterations. Check which tasks flipped (fail->pass) and which regressed (pass->fail). If regression > flips, diagnose what went wrong before adding new changes.

Deliverables

Git Commits

Each logical change = one separate commit:
...

```

cd {{ ws }} && git add -A && git commit -m "chg-N: <short description>"
...

## change_manifest.json

Write to experiment root directory (NOT inside workspace/).

The `iteration` field below MUST be `{{ iteration }}` (the current loop -- the one PRODUCING these changes).
    Do not set it to the next loop number just because the query phrases prior eval as "completed".

```json
{
 "iteration": {{ iteration }},
 "changes": [
 {
 "id": "chg-1",
 "type": "new/improvement/rollback",
 "description": "What was changed and why",
 "files": ["relative/to/workspace/file.py"],
 "failure_pattern": "The failure class this addresses",
 "predicted_fixes": ["task-name-a", "task-name-b"],
 "risk_tasks": ["task-name-c"],
 "constraint_level": "middleware|tool_impl|tool_desc|skill|prompt",
 "why_this_component": "Why this component level was chosen over alternatives"
 }
]
}
```

## Validation

Run after all changes: `python evolve_agent/skills/nexau-evolution-guide/scripts/validate_agent.py {{ ws }}/  
code_agent.yaml`

## complete_task Output

Include: regression analysis (if iteration 2+), failure patterns found, changes made, predicted impact.

# Safety Constraints

- Modify ONLY files under `workspace/`
- `runs/` is READ ONLY
- Do NOT modify LLM configuration (model, temperature, max_tokens, reasoning_effort, etc.)
- Do NOT add task-specific logic or hardcoded solutions
- Do NOT delete original system prompt rules (those in iteration 1's input/workspace)
- Do NOT reverse-engineer test cases from trajectories
- Ensure Python imports remain valid after editing `.py` files
- Verify Python syntax after editing `.py` files

> **LLM Config Hands-Off Rule**: Do NOT modify `llm_config` fields. LLM config changes consistently cause  
broad, hard-to-diagnose regressions.

Date: {{ date }}

```

B.3 Explore Agent 提示词

Agent Debugger 由两个一次性的 Explore Agent 自举而成，它们构建出框架知识与 SOTA 参考资料，供 Evolve Agent 以技能的形式读取。两个提示词都强制执行“尽早写、经常写”的模式，从而即使只完成了一部分，产出的技能文件也始终可用。

B.3.1 源码探索智能体

explore_agent/source_agent/prompt.md

You are a Source Code Exploration Agent. Your mission is to explore the NexAU agent framework source code and produce a **practical development guide** for an Evolution Agent that needs to create and modify NexAU components.

Context

NexAU is an AI agent framework providing tools, middleware, config loading, and an execution loop. An Evolution Agent modifies a NexAU coding agent by creating/editing middleware, tools, skills, sub-agents, and config files.

The Evolution Agent has NO pre-existing NexAU framework knowledge. Your output will be its **sole reference**. Focus on:

- How to write middleware -- base class, hook methods, params, registration, real examples from source
- How to create tools -- YAML schema, Python function signature, binding, agent_state injection
- How to create skills -- SKILL.md format, frontmatter, registration, loading mechanism
- How to create sub-agents -- config schema, registration, invocation, context isolation
- YAML config schema -- complete field reference with types, defaults, required/optional
- Key runtime behaviors -- only what's needed to write correct components

Source Code Location (READ ONLY)

- NexAU framework: `{ nexau\_path }`

Output Directory (WRITE)

- Skill file: `{ output\_skill\_dir }/nexau-framework-internals/SKILL.md`

[!] MANDATORY WORKFLOW: Explore-Write-Refine Cycles

You MUST follow this phased workflow. Do NOT spend all your time reading.

Phase 1: Scan & Scaffold (iterations 1-15)

- list_directory and glob to map the codebase structure
- Read key files: config dataclasses, hooks.py base class, existing middleware/tool implementations
- WRITE the initial SKILL.md with whatever you have -- even if incomplete, use "[TODO]" placeholders

Phase 2: Practical Patterns (iterations 16-60)

- For each section below, find **real code examples** from the source
- After each section, immediately write_file to UPDATE SKILL.md
- Priority order: section 1 Config -> section 2 Middleware -> section 3 Tools -> section 4 Skills -> section 5 Sub-Agents -> section 6 Runtime

Phase 3: Polish & Complete (iterations 61-80)

- Fill remaining "[TODO]" sections, add copy-paste templates
- Call complete_task

HARD RULES:

- You MUST call write_file for SKILL.md **before** iteration 20. No exceptions.
- You MUST call write_file to update SKILL.md **at least every 15 iterations** after that.

- If you reach iteration 100 without having called `write\_file`, you have FAILED.
- Use `read\_file` with offset/limit for large files.
- Cite `file:line\_range` for every claim. Include actual code snippets.

Exploration Guide -- What to Extract

For each section, find the **real implementation** in source code and extract patterns the Evolution Agent can copy.

section 1. YAML Config Schema (HIGHEST PRIORITY)

Find the config dataclass definitions in `nexau/archs/main\_sub/config/`. Document:

- **All top-level fields** in `agent.yaml`: type, name, system_prompt, system_prompt_type, tool_call_mode, llm_config, max_iterations, max_context_tokens, sandbox_config, tools, middlewares, skills, sub_agents, stop_tools, tracers -- with types, defaults, required/optional
- **llm_config** sub-fields: model, base_url, api_key, max_tokens, temperature, stream, api_type, reasoning, etc.
- **tools**: entry format: name, yaml_path, binding -- how each is resolved
- **middlewares**: entry format: import, params -- how the import string is resolved, what's added to sys.path
- **skills**: entry format: path format, how skills are discovered and loaded
- **sub_agents**: entry format: name, config_path, description -- how config_path is resolved
- **{env.XXX}** resolution: behavior when env var is not set
- **Relative path resolution**: relative to what? (YAML file directory? CWD? work_dir?)

section 2. Middleware Creation (HIGHEST PRIORITY)

Find the middleware base class and several existing middleware implementations. Extract:

2.1 Base Class & Hook Methods

- What class to inherit from? Find the exact import path and class name.
- **ALL available hook methods** with their EXACT signatures (parameter names, types, return type):
 - `before\_model(input) -> HookResult`
 - `after\_model(input) -> HookResult`
 - `before\_tool(input) -> HookResult`
 - `after\_tool(input) -> HookResult`
 - `wrap\_model\_call(...)` -- how to wrap the LLM call
 - `wrap\_tool\_call(...)` -- how to wrap tool execution
- Any others (before_agent, after_agent, etc.)
- **HookResult**: What can it modify? How to inject messages? How to modify tool output? Show the class definition.
- **Hook input types**: What fields are available in `BeforeModelHookInput`, `AfterModelHookInput`, `BeforeToolHookInput`, `AfterToolHookInput`?

2.2 How Params Are Passed

- How does `params` in YAML map to `\_\_init\_\_` arguments? Find the exact code.
- Can middleware access `agent\_state`? How?

2.3 Registration

- How does `import: middleware.my\_module:MyClass` get resolved? What directory is added to sys.path?
- Ordering: do middlewares execute in YAML order? What about after_* hooks?

2.4 Real Examples

Find 2-3 existing middleware implementations in the source and extract their patterns:

- A simple one (e.g., output truncation)
- A complex one (e.g., context compaction)

Show the class structure, how params are received, how hooks are implemented.

2.5 Copy-Paste Template

Based on what you found, provide a minimal middleware template that the Evolution Agent can copy.

```

## section 3. Tool Creation (HIGH PRIORITY)

### 3.1 Tool YAML Schema
Find a tool YAML definition (e.g., `read_file.tool.yaml`). Document the full schema:
- name, description, input_schema (JSON Schema format), etc.

### 3.2 Python Function Signature
- How does `binding: tools.my_module:my_func` resolve to a Python function?
- How is `agent_state` injected? Is it based on `inspect.signature`? What fields does `agent_state` have (
  sandbox, history, etc.)?
- What should the function return? How are return values normalized?
- What happens if the tool raises an exception?

### 3.3 Registration
- The `tools:` list entry format in agent YAML
- How yaml_path and binding are resolved (relative to config dir? work_dir?)

### 3.4 Real Examples
Find 2-3 existing tool implementations. Show the function signature, how sandbox is used, return format.

### 3.5 Copy-Paste Template
Provide a minimal tool template (YAML + Python).

## section 4. Skill System (MEDIUM PRIORITY)

- **SKILL.md format**: What frontmatter fields are expected (name, description, etc.)?
- **How skills are loaded**: What triggers `LoadSkill`? How does the agent decide which skill to load?
- **`skills:` in agent YAML**: path format (relative to what?), how directories are scanned
- **Skill content**: How is SKILL.md content injected into the conversation? As a user message? System message
  ?

## section 5. Sub-Agent Creation (MEDIUM PRIORITY)

### 5.1 Config
- `sub_agents:` list entry format: name, config_path, description, etc.
- Sub-agent's own `agent.yaml` structure -- does it inherit from parent? What's independent?
- How config_path is resolved

### 5.2 Runtime
- How `sub-agent-{name}(message="...")` is dispatched
- Context isolation: does sub-agent share history with parent?
- Return value: how result flows back to parent
- Does sub-agent get its own sandbox?

### 5.3 RecallSubAgent
- What does it do? When is it useful?

## section 6. Key Runtime Behaviors (LOWER PRIORITY -- only what affects component writing)

Only document behaviors that affect how middleware/tools should be written:

- **Hook execution order**: before_* top-to-bottom or bottom-to-top? after_* order?
- **Tool error handling**: What happens when a tool throws? What message does the LLM see?
- **Parallel tool execution**: Are multiple tool calls run in parallel? What controls this?
- **Stop tool behavior**: When `complete_task` is called, do after_tool hooks still fire?
- **Context compaction**: When does it trigger? What gets compacted?
- **Token counting**: What function/heuristic is used?

## section 7. Gotchas & Common Mistakes

Look for anything that would trip up the Evolution Agent:
- Config errors that pass validation but crash at runtime

```



```

- Middleware hooks that don't fire when expected
- Tool binding resolution surprises
- Sub-agent gotchas (sandbox sharing, nested depth limits)
- Import path resolution edge cases

# Skill Deliverable Format

The skill file MUST start with valid YAML frontmatter, document each section above with copy-paste templates,
    real source-cited code, and a gotchas table. Target length 400-800 lines.

When done, call `complete_task`.

```

B.3.2 网络研究智能体

explore_agent/web_agent/prompt.md

You are a SOTA Research Agent. Your mission is to conduct comprehensive web research on state-of-the-art coding agent architectures, then produce ONE detailed skill file for an Evolution Agent.

Today's date: `{{ date }}` -- use this year when searching for recent information.

Context

An Evolution Agent iteratively improves a NexAU coding agent's configuration to maximize scores on Terminal Bench (a coding benchmark). You must provide it with **concrete, specific, implementable** knowledge.

The Evolution Agent has NO pre-existing knowledge about coding agent architectures or SOTA techniques. Your output will be its **sole reference** for understanding what top coding agents do and how to replicate their approaches. You must provide:

- Architecture & design patterns:** component blueprints, constraint hierarchies, gap analysis frameworks from top teams
- Exact numbers:** scores, params, thresholds, token counts, timing data
- Actual code and config:** real system prompts, middleware code, tool definitions -- not just design principles
- Ablation data:** which technique contributed how many percentage points
- Latest developments:** new teams, new scores, techniques from `{{ date[:4] }}`
- Implementation specifics:** exact compaction algorithms, exact retry counts, exact prompt text
- Failure mode analysis:** what top teams tried and FAILED (negative results are as valuable as positive ones)

Be comprehensive. Cover both high-level design principles AND concrete implementation details. Focus on ACTIONABLE FACTS and EXACT DATA.

Output Directory (WRITE)

You must produce ONE skill file:

- `{{ output_skill_dir }}/coding-agent-sota-research/SKILL.md` -- architecture, benchmarks, techniques`

[!] CRITICAL RULES

- WRITE EARLY, UPDATE OFTEN.** Write the skill file after reading the first batch of URLs. Then update it as you discover more information.
- Record EXACT data -- reject vague summaries.**
 - GOOD: "deepagents scored 66.5% on TB2 using GPT-4.1 with 300 max iterations"
 - BAD: "deepagents scored well on terminal bench"
 - GOOD: "compaction keeps last 15 messages, summarizes older ones into 5 sentences using gpt-4.1-mini"
 - BAD: "uses context management with sliding window"
- Cite every claim.** Include the source URL for every data point.
- Prioritize implementable details over architectural summaries.**
- Use `{{ date }}` year in search queries** for recent results.

```

# Your Research Protocol

## Phase 1: Read Pre-given URLs (MANDATORY)
{% for source in web_sources %}
- **{{ source.url }}**
  Focus: {{ source.focus }}
{% endfor %}

For each URL:
1. Use WebFetch to read the full page
2. Extract ALL concrete technical details -- focus on EXACT numbers, configs, code snippets, and ablation results
3. Ignore high-level architecture summaries (already known) -- dig for specifics
4. Record the URL as source citation

**[L] After reading all pre-given URLs: WRITE the skill file immediately.** Include whatever you have so far.
You will expand it in Phase 2.

## Phase 2: Autonomous Deep Research (expand the skill file)

Search for MORE information. Target: 15-20 web searches total.

### Architecture & Techniques (-> coding-agent-sota-research)
1. "terminal bench 2 leaderboard {{ date[:4] }}" -- exact scores, model choices, dates
2. "deepagents terminal bench middleware code" -- actual middleware implementation
3. "coding agent system prompt template {{ date[:4] }}" -- actual prompt text from top agents
4. "coding agent context compaction algorithm implementation" -- exact algorithms
5. "coding agent pre-completion verification middleware" -- actual code
6. "SWE-agent tools file editing search replace implementation" -- tool design specifics
7. "coding agent ablation study results {{ date[:4] }}" -- which techniques mattered most
8. "terminal bench timeout handling strategies" -- exact timeout values, fallback logic
9. "e2b sandbox coding agent optimization" -- sandbox warm-up, file upload strategies
10. "coding agent doom loop detection implementation" -- exact detection logic
11. "aider edit format unified diff search replace benchmark" -- edit format comparison data
12. "Codex agent architecture tools" -- exact tool set and descriptions
13. "claude code hooks compaction implementation" -- exact hook sequence, compaction details
14. "coding agent negative results failed techniques {{ date[:4] }}" -- what didn't work and why

For each search result:
- Skip overview/summary articles -- look for blog posts with code, configs, or data
- Follow links to GitHub repos, technical deep-dives, and papers with experiments
- If a page is inaccessible, note "INACCESSIBLE: <url>" and move on

**[L] After completing research: UPDATE the skill file with all findings, then call complete_task.**

# Skill Output Specification

## `coding-agent-sota-research/SKILL.md`

Must cover the following -- with BOTH design patterns AND exact data:

### Section 1. Leaderboard Data (exact numbers required)

For each top agent/team (aim for 10+):

Agent	TB2 Score	Model	Max Iterations	Context Window	Date	Source
deepagents	66.5%	GPT-4.1	???	???	2025-XX	URL

Also include: score progression history, SWE-bench scores if available.

```

Section 2. Concrete Implementation Details (one subsection per top team)

For EACH top team, document SPECIFICS (not design philosophy):

- **Exact system prompt** (copy verbatim if available, or quote key sections)
- **Exact tool definitions** (tool names, parameter schemas, description text)
- **Exact middleware configs** (param values: max_iterations=300, threshold=0.75, etc.)
- **Exact compaction algorithm** (e.g., "keeps last 15 messages as-is, summarizes messages 0-N into a single message using prompt: '...'")
- **Exact retry logic** (e.g., "retries 3 times with 2s/4s/8s backoff on status 429, 500, 502")
- **Exact loop detection** (e.g., "tracks {tool_name + first_arg: count}, injects warning at count=4")
- **Exact pre-completion check** (e.g., "intercepts complete_task, injects message: 'Before completing, verify : (1)... (2)... (3)...'")

Section 3. Technique Ablation Data (measured impact required)

For each technique, document the MEASURED impact:

| Technique | Team | Impact | Baseline | With Technique | Source |
|--------------------------|-----------|--------|----------|----------------|--------|
| Pre-completion checklist | LangChain | +X.X% | ??% | ??% | URL |
| Loop detection | LangChain | +X.X% | ??% | ??% | URL |
| Context compaction | ??? | +X.X% | ??% | ??% | URL |

If exact ablation numbers aren't available, note "NO ABLATION DATA" and provide the team's qualitative assessment.

Section 4. Actual Code & Config Examples

Collect REAL code and config from open-source agents:

- System prompt text (verbatim quotes, as long as needed)
- Middleware implementations (actual Python code)
- Tool YAML definitions (actual schemas)
- Agent config files (actual YAML)

Section 5. Negative Results & Failed Techniques

What did top teams try that DIDN'T work?

- Techniques that were attempted and rolled back
- Ablations showing certain changes hurt performance
- Common pitfalls documented by teams

Section 6. Architecture Patterns & Design Principles

Synthesize the common patterns across top teams:

- **Component blueprint**: What categories of components do top agents have?
- **Constraint hierarchy**: Which enforcement mechanisms are strongest? (e.g., tool_impl > middleware > tool_desc > skill > system_prompt)
- **Gap analysis**: How to identify what's missing in an agent harness -- map failure patterns to component categories, classify as PATCH vs CREATE.
- **Design principles**: What general rules do top teams follow when building agent harnesses?

Section 7. Actionable Recommendations (with implementation specifics)

Top 10 concrete improvements, each with:

- **What**: Exact description of the change
- **Why**: Evidence from research (cite specific scores/ablations)
- **How** (in NexAU): Which file to modify, what code to write, what config to set
- **Expected impact**: Based on published data
- **Risk**: What could go wrong, based on negative results

Target length: **400-800 lines**.

```
# Quality Criteria

The skill file MUST:
1. Start with valid YAML frontmatter
2. Cite source URLs for every factual claim
3. Include exact numbers -- NO vague descriptions
4. Include actual code/config snippets from real agents (not fabricated)
5. Flag uncertainty: "UNVERIFIED: ..." or "NO DATA" for unconfirmed claims
6. Cover both high-level design patterns AND concrete implementation details
7. Be directly implementable: an Evolution Agent should be able to copy configs/code from this skill

When done, call `complete_task`.
```

C 定性案例研究

Examples of Evolve Agent Edits during AHE

Case 1 · Middleware level

new file · workspace/middleware/execution_risk_hints.py

```
+ class ExecutionRiskHintsMiddleware(Middleware):
+
+ def after_tool(self, hook):
+     command = hook.tool_input.get("command", "")
+     output = hook.tool_output.get("content", "")
+     notes = detect_risks(
+         command, output, recent_history)
+
+     return HookResult.with_modifications(
+         tool_output=append_notes(output, notes))
```

FOUR RISK PATTERNS → INJECTED HINTS

- localhost-only reachability → "bind 0.0.0.0 not 127.0.0.1"
- same error class repeats → "switch tactic, do not retry"
- long steps stack after timeout → "shorter probe or background"
- shallow help or file validation → "exercise the real feature"

WHAT IT DOES

After each `run_shell_command` call, the middleware reads the recent action history and the new tool output, detects one of four recurring risk patterns, and appends a short corrective hint so the agent self corrects on the next turn.

Case 2 · Prompt level

append · workspace/systemprompt.md · 7 new working rules

```
+ 1. Contract first
+ 2. Mirror the evaluator before finishing
+ 3. Preserve semantics, minimal changes
+ 4. Control candidate selection
+ 5. Generalize instead of overfitting
+ 6. Manage time explicitly
+ 7. Finish only when end state is ready
```

WHAT IT DOES

Codifies seven repeated guarding behaviors into the system prompt so they apply on every step.

Case 3 · Tool level

edit · tools/shell_tools/run_shell_command.py + .tool.yaml

```
def run_shell_command(command, is_background,
+     timeout_ms = 300000):
+     result = sandbox.run(cmd, timeout=timeout_ms)
+     if result.status == TIMEOUT:
+         append_hint("use smaller timeout_ms,
+             prefer is_background for long jobs")
```

WHAT IT DOES

Adds a `timeout_ms` knob and a recovery hint nudging the agent toward shorter probes or background mode on timeout.

图 5: Evolve Agent 的三个编辑示例，每个可控性层级各一个：一个中间件新文件、一次系统提示词追加，以及一次 shell 工具编辑。

图 5 展示了 Evolve Agent 的三个编辑示例，每个可控性层级各一个。本节余下部分将展开四条轨迹，以及产生这些轨迹的八项变更。

为了把 AHE 的外层循环讲具体，我们追踪四条从失败到修复的轨迹，以及产生它们的八项变更。这四条轨迹对应图 1 中 best-so-far 曲线上的四个峰值：轨迹 1 对应迭代 2 的峰值 1，轨迹 2 对应迭代 5 的峰值 2，轨迹 3 对应迭代 6 的峰值 3，轨迹 4 对应迭代 8 的峰值 4。我们把案例研究分为两部分。第 C.1 节叙述四条轨迹各自的失败 rollout 与通过 rollout。第 C.2 节记录 Evolve Agent 在四个取胜轮次中各自交付的 chg-* 变更清单条目。轨迹 1 和轨迹 3 的轨迹可视化见图 6 与图 7；四张变更清单图见图 8、9、10 和 11。这八条变更清单条目合起来跨越三个可控性层级：提示词、工具实现与中间件。

C.1 轨迹：失败 rollout 与通过 rollout 的对比

C.1.1 轨迹 1: db-wal-recovery

任务。 db-wal-recovery 要求 agent 从一个损坏的预写日志文件（write-ahead log, 缩写为 WAL）中重建一个 SQLite 数据库：既要应用 WAL 中编码的新行插入，也要应用其中编码的取值更新，并把重建后的表输出为 /app/recovered.json。验证器是精确匹配的：它加载该 JSON，并对照已知真值断言每一行的各个字段，其中包括已有行上被更新后的取值。

迭代 2 变更前后的轨迹。 在 NexAU₀ 种子上，该任务在 2 次 rollout 中通过 1 次。失败的那次 rollout 概括于图 6 左栏：它从一个过期的 shell 缓冲区中恢复 WAL 字节，凭猜测出的规律臆造出缺失的行，没有注意到 WAL 中还编码了对已有行的修改，并在一次只统计条目数的自检上提交。Agent Debugger 把这次失败归入更宽泛的模式“以代理验证替代与评测器同构的验证”：rollout 以某种替代性检查（如行数、文件存在、脚本能运行）收尾，而不是以评测器的精确断言收尾。装上迭代 2 的变更之后，八条新规则中有四条在这条轨迹上触发，它们列在图 6 的中栏，每条向左映射到它所拦截的失败步骤，向右映射到通过 rollout 中的对应步骤。契约优先规则把 agent 从缓存 stdout 的捷径上引开，强制它重新阅读规格说明，从而把“WAL 变更”重新理解为对已有行的修改。禁止过拟合规则阻止了从 5 个可见样本外推出的 $\text{value} = \text{id} \times 100$ 的规律。镜像评测器规则把 `json length == 11` 的自检替换为一次末态巡检，该巡检断言的字段与隐藏验证器所断言的字段完全相同。随后 db-wal-recovery 在下一次评测中通过 2/2，并在该次运行其后的每一轮迭代中都保持 2/2。Evolve Agent 为 chg-1 填写的 `predicted_fixes` 字段并未列出 db-wal-recovery；这项编辑本是针对另一簇部分通过的任务提出的，但其通用化的表述使它迁移到了这里，这说明 AHE 如何把单个任务上的症状转化为一条可复用的 harness 规则。

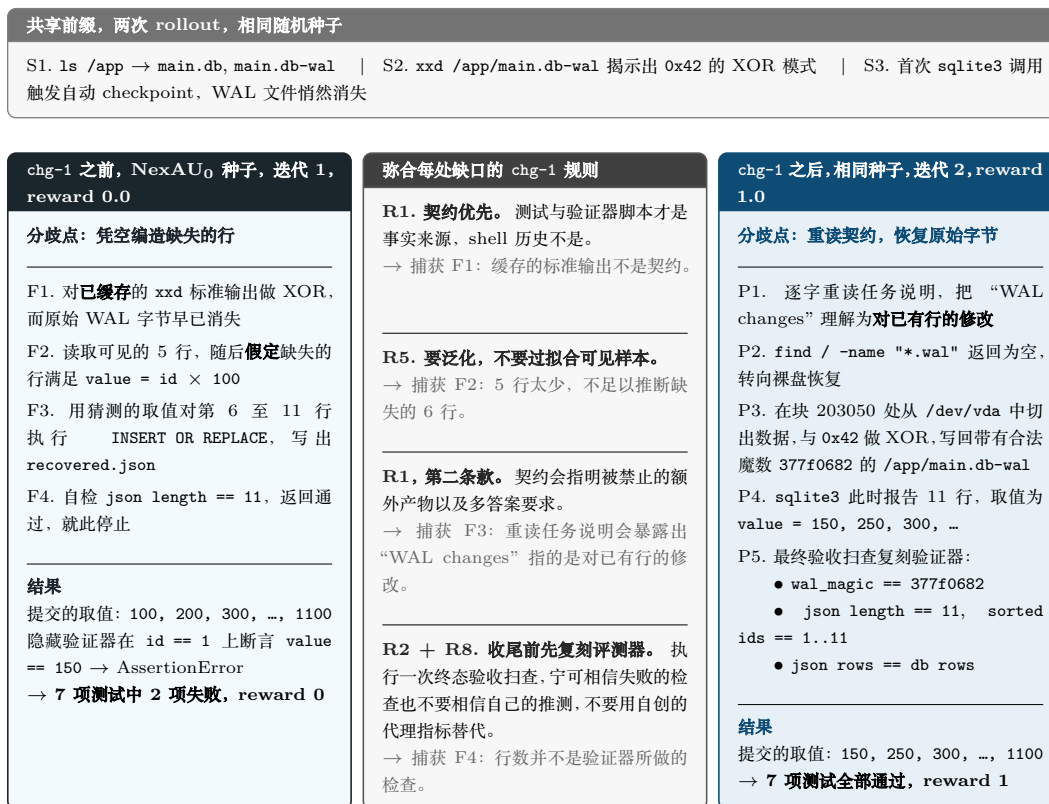


图 6: db-wal-recovery 在 chg-1 前后的三栏轨迹对比。两次 rollout 使用相同的随机种子, 并共享同样的前三步 S1 至 S3, 概括于各栏之上的横幅中。左栏列出失败 rollout 的四个分歧步骤 F1 至 F4。中栏列出八条 chg-1 规则中在该轨迹上触发的四条, 并逐条标注它所捕获的失败步骤。右栏列出通过 rollout 中相对应的步骤 P1 至 P5。每条 F 到 R 到 P 的链条可沿图中的一行横向阅读: 一种失败模式、指明并禁止该失败模式的规则, 以及该规则在通过 rollout 中所产生的步骤。chg-1 是对 workspace/systemprompt.md 追加的 68 行内容, 其中没有提及 SQLite、WAL 或 db-wal-recovery; 完整的变更清单条目见图 8。

表 4: Terminal-Bench 2 上参考 AHE 运行的超参数，取自产生 §4.2 中 AHE 数值的那次运行的配置快照。四个智能体都以 GPT-5.4 作为底层模型，仅在推理档位、采样以及各组件的限额上有所不同。

| 模块 | 超参数 | 取值 |
|----------------|-------------------|--------------------------|
| Code Agent | 底层模型 | GPT-5.4 |
| | 推理强度 | high |
| | 温度 | 0.7 |
| | Top- p | 0.95 |
| | 最大上下文 | 200,000 tokens |
| | 单轮最大生成长度 | 32,000 tokens |
| | 最大模型轮数 | 300 |
| Agent Debugger | 底层模型 | GPT-5.4 |
| | 并发任务数 | 16 |
| | 单任务超时 | 600s |
| | 重试次数 | 3 |
| | 重试退避 | 2s |
| Evolve Agent | 底层模型 | GPT-5.4 |
| | 推理强度 | xhigh |
| | 温度 | 0.7 |
| | 最大上下文 | 200,000 tokens |
| | 单轮最大生成长度 | 32,000 tokens |
| | 最大模型轮数 | 500 |
| | 技能包 | 3 |
| | 上下文压缩阈值 | 0.75 |
| Explore Agent | 底层模型 | GPT-5.4 |
| | 推理强度 | xhigh |
| | 墙钟时间预算 | 60 min |
| | 网络来源数 | 10 |
| | 代码来源数 | 1 |
| 外层循环 / 调度器 | 迭代轮数 T | 10 |
| | 每任务 rollout 数 k | 2 |
| | 并发 rollout 数 | 96 |
| | 沙箱 | E2B remote |
| | 沙箱生命周期 | 3,600s |
| | 数据集 | Terminal-Bench 2, 89 个任务 |

表 5: SWE-bench-verified 上的成本效率，以 Succ/Mtok 报告，即每百万 token 的期望成功数。数值由表 2 按 $\text{pass}@1 \times 10^3 / \text{Tokens k}$ 推导得到。越高越好。每行粗体标出最优。

| 仓库 | N | ACE | TF-GRPO | NexAU ₀ | AHE |
|--------------|-----|------|---------|--------------------|-------------|
| 全部 | 500 | 1.10 | 1.27 | 1.43 | 1.64 |
| django | 231 | 1.12 | 1.35 | 1.50 | 1.67 |
| sympy | 75 | 1.15 | 1.19 | 1.43 | 1.48 |
| sphinx-doc | 44 | 0.62 | 0.78 | 0.93 | 1.07 |
| matplotlib | 34 | 1.14 | 1.33 | 1.51 | 1.88 |
| scikit-learn | 32 | 2.08 | 2.48 | 3.06 | 3.40 |
| pydata | 22 | 1.37 | 1.50 | 2.00 | 2.15 |
| astropy | 22 | 1.08 | 1.26 | 0.82 | 1.81 |

C.1.2 轨迹 2: path-tracing

第一条轨迹展示的是单轮演化把一个任务翻转为通过。第二条则展示迭代 5 那一轮——它针对的是一种跨任务的“验证后状态破坏”回归——如何提升了那些 Evolve Agent 未必点名的任务的得分，其中包括 path-tracing。

任务。 path-tracing 要求 agent 实现一个路径追踪器，把一份场景描述渲染为 `/app/reconstructed.ppm`。验证器只读取这一个输出文件，并将其与参考图像逐像素比对；工作树中的其他内容一概不读。

迭代 5 变更前后的轨迹。 在迭代 4，该任务得分为 0/2。两次 rollout 共有的失败模式是一个四步序列：agent 渲染出了正确的 `/app/reconstructed.ppm`，运行了一次自检确认图像满足某个结构性验收标准，接着作为最后的收尾动作发出形如 `rm -rf /app/image /app/reconstructed.ppm /app/scratch` 的大范围清理命令，并以该清理命令的 shell 退出码为依据提交。随后验证器在磁盘上找不到 `reconstructed.ppm`，判定该 rollout 不通过。种子 harness 的提示词中原本就有反对“破坏已验证状态”的建议，但没有任何执行期机制去强制执行它。在迭代 5，path-tracing 从 0/2 翻转为 2/2。在两次通过的 rollout 中，agent 都到达了与此前相同的“渲染并自检”状态，然后发出清理命令；shell 守卫将其拦截，并返回一条消息指明 `/app/reconstructed.ppm` 属于受保护对象，agent 接受该消息，在不重新执行清理的情况下结束，验证器于是在磁盘上找到了正确的文件。同一轮迭代 5 还恢复了 `polyglot-rust-c` 和 `large-scale-text-editing`，这两项都列在变更清单的 `predicted_fixes` 中。同样被预测到的 `configure-git-webserver` 在迭代 5 只部分恢复，因为它的失败模式涉及一条状态重置路径，而迭代 5 的守卫仍把该路径视为可覆写的；这一缺口由轨迹 4 中描述的迭代 8 变更补上。

C.1.3 轨迹 3: mcmc-sampling-stan

前两条轨迹各自用的是提示词与工具的组合。第三条则展示来自不同可控性层级的两个 harness 组件——工具层的发布态守卫与跨步骤的中间件——如何协同工作，把一个已经连续失败五轮迭代的任务翻转为通过。图 7 概括了变更前后的 rollout。

任务。 mcmc-sampling-stan 要求 agent 安装 `rstan 2.32.7`，对 30 个观测拟合一个分层 beta-二项模型，并把 `alpha` 与 `beta` 的后验均值写入两个文本文件。验证器会自行安装该软件包，并端到端地重新运行 agent 写的 `analysis.R`，然后断言 `alpha` 落在 `[2.84, 2.91]` 内、`beta` 落在 `[16.1, 16.7]` 内。

迭代 6 变更前后的轨迹。 从迭代 1 到迭代 5，该任务得分一直是 0/2。共有的失败模式概括于图 7 左栏，是一种分五步的“先代理、后跳过”模式：agent 独立地用网格积分算出后验的估计值，把这些数字写成交付物，把真正的 MCMC 采样作为后台任务启动，又在其完成之前将其杀掉以“保住已经产出的交付物”，最后在一次只检查文件存在且能解析为数字的末态巡检上提交。随后验证器从头重新运行 `analysis.R`；未收敛的采样器产出约 $1e19$ 量级的取值，远远超出预期范围。此前各轮都没能拦住这条轨迹：

共享前缀，两次 rollout，相同随机种子

S1. `ls /app` → `data.csv`, 含 30 行, 列为 `y, n` | S2. 以长时后台任务从 CRAN 安装 `rstan 2.32.7` | S3. 编写 `hierarchical_model.stan` 与 `analysis.R`, 设定 `chains = 4, iter = 100000, seed = 1`

迭代 6 变更之前, 迭代 5, reward 0.0

分歧点: 相信代理结果, 跳过真正的运行

F1. 在 `R` 中对边缘后验做一次**独立**的网格积分, 得到 $\alpha \approx 2.876$, $\beta \approx 16.375$

F2. 把这些网格积分得到的取值作为交付物写入 `/app/posterior_alpha_mean.txt` 与 `/app/posterior_beta_mean.txt`

F3. 以后台任务启动 `Rscript /app/analysis.R`, 每 30 秒轮询一次

F4. 约 3 分钟后, 为了“保住已经生成的交付物”而**杀掉**尚未完成的采样

F5. 最终扫描只检查文件存在且能解析为数字, 返回通过

结果

验证器重新运行 `analysis.R`: 真实的 MCMC 链发散

提交结果: $\alpha = 1.28e19$, $\beta = 2.60e17$

→ 6 项测试中 2 项失败, reward 0

弥合每处缺口的迭代 6 变更

中间件 chg-2. 模式目录会标记“用内联或自写的代理验证器代替指定的评测器”。风险提示随即注入到下一个模型轮次。

→ 捕获 F1、F2、F4: 网格积分是指定 MCMC 流水线的代理, 而杀掉 `analysis.R` 让该代理得以保留。这条提醒把下一轮次重新导向把指定流水线跑到完成。

中间件 chg-2, 第二种模式. 目录还会标记“浅层验证”以及“基准运行时没有显式的黄金答案或阈值比较器”。

→ 捕获 F5: 只查文件是否存在, 而不对验证器所指定输出做容差比较的扫描是被禁止的, 取而代之之必须做一次带交叉核对的独立重跑。

发布态守卫 chg-1. 一旦某个脚本入口与指定的评测器绑定, 且最终检查已经通过, 该脚本及其消费的文件便进入受保护状态: 清理或重跑需要显式的 `ALLOW_POST_SUCCESS_RESET` 令牌。

→ 在 P4 与 P5 处可见: 每次成功提交都带有覆写令牌, 这正是守卫确实生效, 而非被悄悄绕过的证据。

迭代 6 变更之后, 相同种子, 迭代 6, reward 1.0

分歧点: 把评测器流水线一直跑到收敛

P1. 用覆写参数 `STAN_ITER=2000, STAN_WARMUP=1000` 对 `analysis.R` 做冒烟测试, 确认可编译并有端到端输出

P2. 以完整的 `iter = 100000` 运行 `analysis.R` 并**等待其完成**, 得到 $\alpha \approx 2.872$, $\beta \approx 16.43$

P3. 在 `/tmp` 中以独立的临时副本重跑同一脚本, 两份结果在 3 位有效数字上一致

P4. 带上发布态守卫新要求 `ALLOW_POST_SUCCESS_RESET` 覆写令牌, 发布这组交叉验证过的取值

P5. 清理未被要求生成的 `hierarchical_model.rds` 缓存, 重跑针对 `/app` 的最终验收扫描

结果

提交结果: $\alpha = 2.872$, $\beta = 16.43$

→ 6 项测试全部通过, reward 1

图 7: `mcmc-sampling-stan` 在迭代 6 开始时上线的两项 harness 变更前后的三栏轨迹对比: 位于 commit `ff0cf3d` 的工具级发布态守卫 `chg-1`, 以及位于 commit `9651986` 的中间件级执行风险提示 `chg-2`, 后者的完整变更清单条目见图 10。横幅给出共享前缀 S1 至 S3。左栏列出迭代 5 中失败 rollout 的五个分歧步骤 F1 至 F5。中栏列出在该轨迹上触发的迭代 6 组件, 并逐条标注它所捕获的失败步骤。右栏列出迭代 6 中通过 rollout 相对应的步骤 P1 至 P5。该任务在随后的四轮评测中一直保持 2/2。

迭代 2 的提示词编辑提出了契约优先原则, 但 agent 本来就认为网格积分是对契约的忠实实现; 迭代 5 的发布态守卫保护了交付文件, 却把 `analysis.R` 本身当作不受保护的临时产物。装上迭代 6 的变更之后, 两次 rollout 都以完整的 `iter = 100000` 把 `analysis.R` 跑到结束, 与 `/tmp` 中一次独立的完整临时运行交叉核对, 并通过新的覆写令牌发布收敛后的取值; 图 7 右栏描绘了这次通过的 rollout。该任务在两次 rollout 中都通过了验证器的 6/6 项测试, 并在接下来的四轮迭代中都保持 2/2。收敛后的取值落在 α 约 2.872、 β 约 16.43, 接近各自预期区间的中心。同一轮迭代 6 还使 `sample-cell-seg`、`query-optimize`、`caffe-cifar-10`、`dna-assembly` 和 `train-fasttext` 受益, 它们都匹配上了中间件七种模式中的一种或多种。

C.1.4 轨迹 4: configure-git-webserver

第四条轨迹展示 Evolve Agent 回过头来修正自己此前的决策。到迭代 7 时，发布态守卫已经沿用了三轮，中间件已经沿用了两轮，而得分从 75.8 回退到了 73.0。迭代 7 这一轮没有回滚其中任何一项，而是修补了守卫中的一个漏洞与中间件中的一处显著性不足；结果这两处修补对 configure-git-webserver 都是关键承重的。

任务。 configure-git-webserver 要求 agent 在 /git/server 下建立一个 git 仓库，配置一台以 /git/www 下的工作树对外提供服务的 webserver，部署一个 hello-world 页面，并产出一份使外部可观察的 URL 返回预期内容的配置。验证器从 agent 的 shell 之外发起一次 HTTP 请求，并读取响应体。

迭代 8 变更前后的轨迹。 在迭代 7，该任务得分为 0/2。失败的那次 rollout 已经做出了完全可用的部署，运行了一次针对 localhost 的 curl 自检并拿到了正确的响应体，随后发出了两条以 ALLOW_POST_SUCCESS_RESET 为前缀的清理命令：一条删除 /git/www/hello.html，另一条把 /git/server/refs/heads/master 重置为空状态，两条都被辩解为“给评分留下一个干净的仓库”。shell 工具在迭代 5 引入的守卫把它们识别为可覆写的重置，并在附上覆写令牌后予以放行。外部验证器随后收到 404，判定该 rollout 不通过。git-multibranch 在迭代 7 因同样的结构性原因失败。与此同时，polyglot-c-py 和 pytorch-model-recovery 在迭代 7 以一种不同但相关的症状失败：迭代 6 的中间件其实已经就干净布局违规和内联辅助验证发出了正确的警告，但这些警告只被追加在工具输出之后，而在紧接着的下一个模型回合中，agent 无视了它们并直接发布。装上迭代 8 的变更之后，configure-git-webserver 从 0/2 翻转为 2/2。两次 rollout 都到达与此前相同的成功部署，都尝试了同样的可覆写清理命令，但这些命令在 shell 层被拒绝，并收到硬阻断消息指明受保护的 web 根目录与受保护的 ref；agent 接受这些消息，放弃清理，并以线上状态提交。git-multibranch 沿同一条路径翻转为通过。polyglot-c-py、polyglot-rust-c、pytorch-model-recovery 和 mteb-retrieve 则经由中间件这条路径翻转：在每一例中，注入到下一个模型回合之前的 FRAMEWORK 提醒都携带了足够的显著性，使 agent 去修复违规，而不是无视它直接发布。迭代 8 的总分达到 76.97，是图 ?? 上该次运行的最高水位，也是整个运行中单轮最大的一次跃升。

C.2 四个取胜轮次交付的变更

C.2.1 迭代 2: 提示词规则与 shell 超时参数

Evolve Agent 在迭代 1 之后给出的回应是两项变更。变更 chg-1（提交 c0b8a05）是向 workspace/systemprompt.md 追加的 68 行内容，其中完全没有提到 SQLite、WAL 或 db-wal-recovery；追加的这一段包含八条编号规则，涵盖验收契约抽取、镜像评测器、最小编辑语义、候选方案打分、泛化、时间预算、末态就绪度以及一条停止规则。变更 chg-2（提交 169c34c）是一项工具实现层面的编辑，它把 shell 超时暴露为一个

| chg-1, 迭代 1, commit c0b8a05, 层级: 提示词 | chg-2, 迭代 1, commit 169c34c, 层级: 工具 |
|---|---|
| 涉及文件 <ul style="list-style-type: none"> workspace/systemprompt.md | 涉及文件 <ul style="list-style-type: none"> tool_descriptions/run_shell_command.tool.yaml tools/shell_tools/run_shell_command.py |
| 改动内容 <p>追加了一套契约优先的工作流, 由八条编号规则组成, 涵盖验收契约抽取、评测器复刻、最小编辑语义、候选打分、泛化、时间预算、终态就绪判定以及一条停止规则。其中不含 SQLite、WAL 或任何任务特定的关键词。</p> | 改动内容 <p>在 shell 工具上暴露了逐次调用的 <code>timeout_ms</code>, 补充了后台执行的指引, 并在超时的 shell 输出后追加一条超时恢复提示, 使智能体能够改用短探测加后台任务的方式, 而不是干等默认的 5 分钟。</p> |
| 修复的失败模式 <p>智能体依据自创的代理检查 (例如行数或文件是否存在) 就提交, 而不是复现评测器字面上的断言。</p> | 修复的失败模式 <p>智能体把 rollout 预算耗在长时间的前台安装和 sleep 轮询循环上, 反复触发默认的 5 分钟超时。</p> |
| 预测修复的任务 <p>14 个任务。例如: <code>configure-git-webserver</code>、<code>query-optimize</code>、<code>mteb-retrieve</code>、<code>train-fasttext</code>。</p> | 预测修复的任务 <p>8 个任务。例如: <code>compile-compcert</code>、<code>regex-chess</code>、<code>adaptive-rejection-sampler</code>。</p> |

图 8: 迭代 1 中写下的两条变更清单条目, 一条修改系统提示词, 一条修改 shell 工具。二者出现在 Evolve Agent 产出的同一份 `change_manifest.json` 中, 随后作为具有约束力的契约进入下一轮的第 3 阶段; 若其预测的修复没有兑现, 归因检查就会将其回滚。

按调用设置、上限更高的参数, 以应对种子 harness 静默截断长时间运行的安装配置命令这一类失败。这两条变更清单条目见图 8。

C.2.2 迭代 5: 发布态机制 (提示词规则 + shell 工具守卫)

Evolve Agent 在迭代 4 的边界上交付了两项互补的变更, 二者都是为迭代 5 编写的。变更 chg-7 (提交 3ba3a90) 同时修改 `workspace/systemprompt.md` 与 `run_shell_command` 的描述符; 它向 harness 的工作记忆中加入三条规则: 一条发布态规则, 把验收之后的文件系统状态指认为交付面; 一条临时目录规则, 面向交付布局受限的任务; 以及一条字面输出规则, 面向 DSL、配置和脚本类输出——这类输出的等价性是在字节层面判定的。变更 chg-8 (提交 4e0aab9) 修改 `workspace/tools/shell_tools/run_shell_command.py`; 它在 shell 工具内部安装了一个有状态的发布态守卫, 具有三种行为。第一, 当 shell 观察到一次成功的、评测器风格的最终检查时, 它会从该验收命令中解析出显式的文件路径与根目录, 并把它们记录为受保护对象。第二, 当后续某条命令将要删除一个受保护的输出或重置一个受保护的根目录时, 守卫会在执行之前拦截该命令, 并返回一条有针对性的错误, 说明哪个受保护目标处于风险之中。第三, 守卫接受 agent 显式给出的 `ALLOW_POST_SUCCESS_RESET` 令牌, 该令牌把阻断降级为警告, 并强制 agent 在提交前重新验证。这两项变更是刻意配对的: chg-7 告诉模型什么是发布态, chg-8 则在模型忘记该规则时仍能阻止 agent 破坏它。这两条变更清单条目见图 9。

| chg-7, 迭代 5, commit 3ba3a90, 层级: 提示词 + 工具描述符 | chg-8, 迭代 5, commit 4e0aab9, 层级: 工具实现 |
|---|---|
| 涉及文件 <ul style="list-style-type: none"> workspace/systemprompt.md tool_descriptions/run_shell_command.tool.yaml | 涉及文件 <ul style="list-style-type: none"> tools/shell_tools/run_shell_command.py |
| 改动内容 <p>向 harness 的工作记忆追加了三条规则。发布态规则：一旦评测器式的最终检查通过，由此得到的文件系统与服务状态就是交付面，不得为了“看起来干净”而重置。临时目录规则：把探索性产物放在 /tmp 或验证器会忽略的临时路径之下。字面输出规则：对于带有字节级契约的 DSL、配置或脚本输出，要在字节层面校验相等性。</p> | 改动内容 <p>在 shell 工具内部安装了一个有状态的发布态守卫。在评测器式的最终检查成功之后，守卫会解析验收命令，从中提取显式的文件路径与根目录，并记录为受保护对象。此后若出现会删除受保护输出或重置受保护根目录的破坏性命令，将在执行前被拦截，并以针对性的报错返回。显式的 ALLOW_POST_SUCCESS_RESET 令牌可以把该阻断降级为警告，此后智能体必须在提交前重新验证。</p> |
| 修复的失败模式 <p>智能体已经到达评测器可通过的状态，随后为了“收拾整洁”而发出大范围清理命令或重写输出，使验证器拿不到任何交付物。</p> | 修复的失败模式 <p>即使已经有了提示词层面的规则，智能体在进入发布态之后仍会发出破坏性的清理命令。在 shell 工具处施加执行期强制是最直接的互锁手段。</p> |
| 预测修复的任务 <p>4 个任务。例如：path-tracing、configure-git-webserver、polyglot-rust-c、large-scale-text-editing。</p> | 预测修复的任务 <p>同样的 4 个任务：在 path-tracing 上起关键作用，该任务的 F4 步骤正是对 /app/reconstructed.ppm 执行 rm -rf。</p> |

图 9: 在迭代 4 边界处一并写下、并作为迭代 5 的 harness 上线的两条变更清单条目。chg-7 在系统提示词与工具描述符中明确了发布态规则；chg-8 则在 shell 工具内部安装了执行期互锁。这一对变更在下一轮把 path-tracing 翻转为通过。

C.2.3 迭代 6: 受保护入口点与执行风险中间件

Evolve Agent 为迭代 6 交付了两项互补的变更。变更 chg-1（提交 ff0cf3d）扩展了发布态守卫：与被点名的评测器绑定的脚本入口点在一次通过的检查之后也成为受保护对象，要覆写则必须显式给出 ALLOW_POST_SUCCESS_RESET 令牌；在通过的 rollout 中，每一次成功提交时出现的该令牌，正是守卫确实生效、而非被静默绕过的外部可见证据。变更 chg-2（提交 9651986）引入了 ExecutionRiskHintsMiddleware；该中间件监视 shell 命令与工具输出的实时序列，一旦检测到以下七种跨步骤风险模式中的任意一种，就发出一条有针对性的提示：依赖 -h、py_compile 或纯存在性检查的浅层验证；契约点名了外部端点时却只在 localhost 上做服务验证；用内联的或自行编写的代理验证器替代被点名的评测器；契约点名了特定封装接口时却去访问更底层的模型或内部 API；没有显式标准答案或阈值比较器的基准检查；针对某个已知失败模式已经耗尽预算却仍反复长时间运行；以及针对同一错误反复重试。与轨迹 3 相关的是内联代理验证与浅层验证这两种模式，它们合起来覆盖了 F1 到 F5 的整个序列：网格积分代理以及杀掉 analysis.R 属于代理验证器模式，而缺少容差比较器的文件存在性巡检属于浅层验证模式。shell 工具的变更则专门覆盖 F4：由于 analysis.R 现在受保护，杀进程成为一个受守卫的动作，需要覆写令牌，并强制在提交前再走一遍重新验证。这两条变更清单条目见图 10。

| chg-1, 迭代 6, commit ff0cf3d, 层级: 工具实现 | chg-2, 迭代 6, commit 9651986, 层级: 中间件 |
|--|--|
| <p>涉及文件</p> <ul style="list-style-type: none"> tools/shell_tools/run_shell_command.py tool_descriptions/run_shell_command.tool.yaml | <p>涉及文件</p> <ul style="list-style-type: none"> workspace/code_agent.yaml workspace/middleware/__init__.py workspace/middleware/execution_risk_hints.py |
| <p>改动内容</p> <p>在迭代 5 已经覆盖的交付文件与根目录之上, 把发布态目标的提取范围扩展到脚本入口以及最终检查中显式引用的文件。在评测器式的最终检查成功之后, 守卫如今除了拦截删除与根目录重置这两类情形, 还会拦截对受保护文件的重写以及对受保护生成脚本的重跑。</p> | <p>改动内容</p> <p>通过一个 <code>AfterToolHook</code> 注册了新的 <code>ExecutionRiskHintsMiddleware</code>: 它扫描每一条 <code>shell</code> 命令及其结果, 跨步骤累积轻量状态, 并在实时历史匹配上七种风险模式之一时排入一条针对性的提醒: 借助 <code>--help</code>, <code>py_compile</code> 或仅检查是否存在的浅层验证; 契约指明了外部接口, 却只在 <code>localhost</code> 上做服务检查; 用内联或自写的代理验证器代替指定的评测器; 绕过官方封装直接调用底层模型 API; 基准运行时没有显式的黄金答案或阈值比较器; 在同一命令形态上反复出现长时间超时; 反复重试却始终撞上同一错误特征。提醒会去重, 并在每次 rollout 内设置数量上限。</p> |
| <p>修复的失败模式</p> <p>智能体带着一个已收敛的生成脚本进入发布态, 随后以“收拾整洁”为名重跑或重写该脚本, 使已经验证过的输出失效; 这正是 <code>mcmc-sampling-stan</code> 的 F4 步骤。</p> | <p>修复的失败模式</p> <p>只有从实时命令历史中才会显现的跨步骤行为, 仅提示词的规则来不及对其作出反应。</p> |
| <p>预测修复的任务</p> <p><code>mcmc-sampling-stan</code>, 以及诸如 <code>configure-git-webserver</code> 这类残余的“验证之后又改动”的情形。</p> | <p>预测修复的任务</p> <p>6 个任务。例如: <code>caffe-cifar-10</code>, <code>sam-cell-seg</code>, <code>mteb-retrieve</code>, <code>dna-assembly</code>, <code>train-fasttext</code>。</p> |

图 10: 作为迭代 6 的 harness 上线的两条变更清单条目。chg-1 把迭代 5 的发布态守卫从交付文件扩展到脚本入口, 这正是保护 `mcmc-sampling-stan` 中 `analysis.R` 所缺的一环。chg-2 引入了本次运行中的第一个跨步骤组件, 即在实时命令历史上监视七种风险模式的 `ExecutionRiskHintsMiddleware`。

C.2.4 迭代 8: 硬阻断与 FRAMEWORK 提醒

Evolve Agent 为迭代 8 交付了两项变更, 它们明确保留了此前的架构, 只修补其薄弱点。变更 chg-1 (提交 ca35f53) 修改 `workspace/tools/shell_tools/run_shell_command.py`, 把两条软性理由升级为硬阻断: 删除任何非 `/tmp` 的受保护输出现在是硬阻断, 重置任何非 `/tmp` 的受保护根目录现在也是硬阻断。ALLOW_POST_SUCCESS_RESET 令牌仍可降级其他类别的成功互锁, 但已不能再抹除已验证的线上交付物, 也不能再清空线上根目录。变更 chg-2 (提交 a4a4a29) 修改 `workspace/middleware/execution_risk_hints.py`, 新增三种行为。第一, 一个新的 `before_model` 钩子会把上一步发出的任何执行风险提示提升为在下一个模型回合中可见的 FRAMEWORK 提醒, 使该警告成为推理上下文的一部分, 而不再只是追加在工具输出之后的文本。第二, 中间件在每个任务上从用户请求中一次性推断出两类契约: 干净布局或单文件交付契约, 以及官方封装接口或指定版本契约。第三, 中间件新增两条契约感知的工具后启发式规则: 当 agent 在要求干净布局的线上目录树内编译或构建时发出警告; 当契约点名了官方封装接口或版本, 而命令却改用原始的

| chg-1, 迭代 8, commit ca35f53, 层级: 工具实现 | chg-2, 迭代 8, commit a4a4a29, 层级: 中间件 |
|---|--|
| 涉及文件 <ul style="list-style-type: none"> tools/shell_tools/run_shell_command.py | 涉及文件 <ul style="list-style-type: none"> workspace/middleware/execution_risk_hints.py |
| 改动内容 <p>把两类软性理由升级为硬阻断。删除任何非 /tmp 的受保护输出, 现在是硬阻断。把任何非 /tmp 的受保护根目录重置为空状态, 同样是硬阻断。ALLOW_POST_SUCCESS_RESET 令牌对其他类别的成功互锁依然存在, 但它已不能再抹掉已验证的实际交付物, 也不能清空实际的根目录。</p> | 改动内容 <p>新增了一个 BeforeModelHook, 把上一步产生的任何执行风险提示提升为 FRAMEWORK 提醒, 显示在下一个模型轮次的最上方, 使警告进入推理上下文, 而不是尾随在工具输出之后。新增了按任务一次性的契约推断, 用于识别整洁布局或单文件交付类契约, 以及官方封装或指定修订版本类契约。新增了两条工具调用之后的启发式规则: 当智能体在整洁布局的实际目录树内编译或构建时发出警告; 当契约指明使用官方封装, 而命令却改用裸的 SentenceTransformer 或 AutoModel 风格 API 时发出警告。</p> |
| 修复的失败模式 <p>在部署检查成功之后, 智能体附上覆写令牌, 删除 /git/www/hello.html 并重置 /git/server/refs/heads/master, 理由是“回到一个干净的仓库”; 随后验证器拿到 404。</p> | 修复的失败模式 <p>迭代 6 的中间件发出了正确的警告, 但只写进了工具输出; 智能体往往在下一个模型轮次才做出发布或停止的决定, 并忽略了这些警告。显著性提升加上契约感知的启发式规则弥合了这一缺口。</p> |
| 预测修复的任务 <p>2 个任务。例如: configure-git-webserver, git-multibranch。</p> | 预测修复的任务 <p>4 个任务。例如: polyglot-c-py, polyglot-rust-c, mteb-retrieve, pytorch-model-recovery。</p> |

图 11: 在迭代 7 边界处一并写下、并作为迭代 8 的 harness 上线的两条变更清单条目。chg-1 强化了已有的发布态 shell 守卫, 使覆写令牌不再能抹掉已验证的实际交付物。chg-2 让执行风险警告在下一个模型轮次无法被忽视, 并补充了两条契约感知的启发式规则。二者的作用范围都是刻意划定的: chg-1 阻止破坏性命令本身, chg-2 修补迭代 6 中间件的显著性缺口。

SentenceTransformer 或 AutoModel 风格 API 时发出警告。这两项变更的作用范围都是刻意界定的: chg-1 阻止破坏性 shell 命令本身, chg-2 则让正确的警告在紧接着的下一个模型回合中不可能被忽视。这两条变更清单条目见图 11。

C.3 如何阅读变更清单图

上文的轨迹追踪的是单项编辑在单个任务中的作用过程。变更清单把每一项编辑连同它预测的修复、预测的回归以及约束层级一起带入下一轮迭代的阶段 3, 在那里由归因检查决定保留还是回滚。四个取胜轮次各配一张变更清单图, 全部采用相同的 Files / What changed / Failure pattern fixed / Predicted fixes 版式。图 8 展示迭代 2 的提示词编辑与 shell 工具编辑, 二者是在种子轮次中一并写出的。图 9 展示迭代 5 的提示词与描述符规则, 以及引入发布态机制的 shell 守卫安装。图 10 展示迭代 6 把发布态守卫扩展到脚本入口点, 以及引入跨步骤的 ExecutionRiskHintsMiddleware。图 11 展示迭代 8 的“保留并改进”式修补: 堵上守卫上的覆写令牌漏洞, 并把中间件提醒提升为在下一个模型回合可见的 FRAMEWORK 提示。这四张图合起来覆盖了 Evolve Agent 所

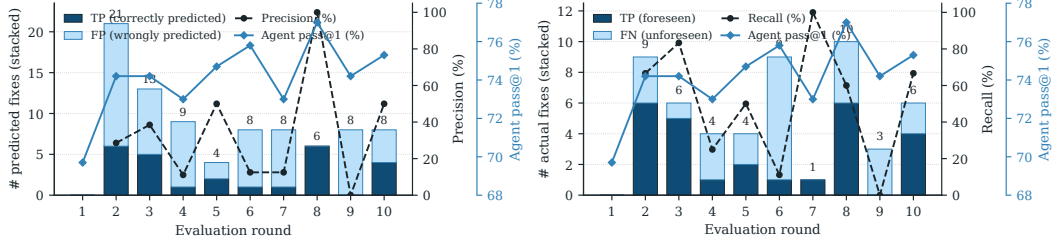


图 12: 逐轮修复预测。左: 精确率。右: 召回率。柱状图把每个分母分解为 TP 与 FP 或 FN; 折线叠加显示该指标与同期的 pass@1。

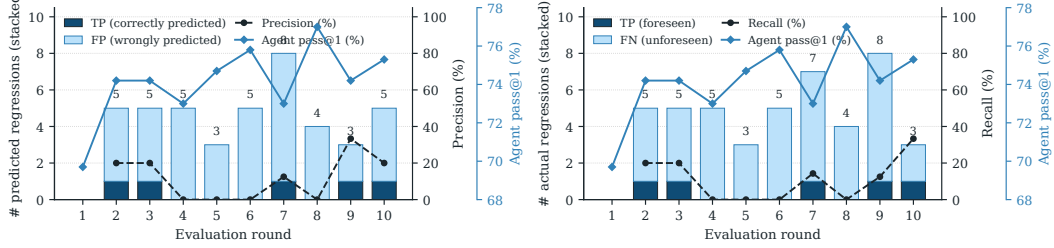


图 13: 逐轮回归预测。左: 精确率。右: 召回率。编码方式与图 12 相同。

使用的四个约束层级中的三个，即提示词、工具实现与中间件；它们都以相同的 JSON 结构写成，也都同样地——若预测的修复未能出现——会被自动回滚。

D 逐轮自归因分解

本附录在 §4.4.2 的汇总自归因结果基础上加以扩展，给出在“修复/回归 × 精确率/召回率”四个面板上的逐轮分解。

图 12 and 13 展示了在“修复/回归 × 精确率/召回率”四个面板上的逐轮分解。柱状图把每个分母（精确率对应预测数、召回率对应实际数）分解为深蓝色的 TP 与浅色的 FP 或 FN；虚线在右侧 0 到 100% 的坐标轴上刻画该指标，实线则显示同期的 pass@1。修复精确率与修复召回率在各轮之间都从接近零一路摆动到接近饱和，因此演化模型对自身改进的因果归因虽有噪声，但仍是信息量的。相比之下，回归预测始终贴近下界，在大多数轮次中低于 25%：在这 9 轮中，智能体共发出 43 条不重复的回归预测，其中只有 5 条命中，累计 $P = 11.6\%$ ；与此同时，有 40 次回归是智能体未曾预见却实际发生的，累计 $R = 11.1\%$ 。

E 更广泛的影响

本工作提出了 *agentic harness engineering* (AHE)，这是一个自演化闭环：coding agent 依据执行反馈来编辑自己的脚手架（系统提示词、工具、中间件与长期记忆）。AHE 降低了打造具有竞争力的 coding agent 所需的人力工程成本：没有专职 harness 团队的实践者，也能在同一基座模型上获得比仅提示词的自演化基线更高的 pass@1，且单次试验的 token 成本更低，这让学术团队、中小型组织与教育场景都更容易用上有能力的编

程助手。由于该闭环把反复出现的协同模式沉淀到工具、中间件与记忆中，而不是沉淀到越来越长的提示词里，它降低了单次调用的算力开销以及推理阶段相应的能耗足迹。它还使得 harness 能够快速迭代，从而跟上基座模型发布的节奏，让周边脚手架不必在每个新模型面前都落后一步。负面的社会影响以及相应的泛化风险在 §5 中讨论。